Pramana — The Complete Project Guide

Written 2026-10-15 and revised 2026-10-18 (IL-TUR test split added; every mechanism re-checked against the code; stale figures corrected) from the project's own records (git history, `CLAUDE.md`, the `stuff/docs/` evaluation files, the code). Every number here can be traced to one of those files; where a number lives is given in brackets so you can show it if asked. Read Part A once end to end, then use Part F (the question bank) to rehearse. Part B now gives each stage's exact parameters (B14 collects them in one table), so you can answer "how many / how long / what threshold" without opening the code.

How to use this guide

If you have...	Read
10 minutes	A1 (the pitch), A3 (the worked example), E1 (the numbers card)
1 hour	All of Part A and Part B, then E1
An evening	Everything, then answer Part F out loud without looking

Contents

- **Part A — What it is**: the pitch, the problem, what a user sees, the architecture
- **Part B — How it works**: every stage in detail, the tech stack and why each piece was chosen
- **Part C — The data**: every dataset, how it was cleaned, what went wrong with it
- **Part D — How it was built and tested**: the timeline, evaluation, every result
- **Part E — Problems, lessons, limits**: what went wrong and what it taught, numbers card, glossary
- **Part F — Question bank**: 130 questions, simple to hostile, with answers

---

Part A — What it is

A1. The pitch

One sentence. Pramana is a research assistant for Indian criminal law that reads a case's facts, works out which code applies (the old Indian Penal Code or the new Bharatiya Nyaya Sanhita), suggests the sections the facts disclose, **checks every section number against a database built from the real statute text before showing it**, and then finds precedent judgments, drafts defence counter-arguments and shows how past appeals on those sections ended, all running locally on a laptop with no cloud and no cost.

Thirty seconds. Lawyers in India have had a confusing year: on 1 July 2024 the Indian Penal Code (IPC, 1860) was replaced by the Bharatiya Nyaya Sanhita (BNS, 2023), and every section number changed. Theft was IPC 378/379; it is now BNS 303. Murder was IPC 302; BNS 302 is now about wounding religious feelings. An offence committed before that date is still tried under the IPC. AI chatbots make this worse because they invent or mix up section numbers. Pramana fixes the number problem structurally: a deterministic rule picks the code from the offence date, a small local language model proposes sections, and **nothing it proposes is shown as real unless it exists as a node in a citation graph built from the ingested statutes**. A second check asks whether the facts actually support each section. Then it pulls matching judgments, writes counter-arguments grounded only in verified material, and counts appeal outcomes.

Two minutes — add: it is retrieval-augmented generation (RAG), not a fine-tuned model; the embedder is BGE-M3, chosen by a measured bake-off over the legal-domain InLegal-Sbert; the corpus is 87,969 chunks from about 2,750 Supreme Court and High Court judgments plus both codes; the LLM is Qwen 3.5 4B running on Ollama on a 4 GB laptop GPU; the orchestration is a LangGraph state machine with a retry loop; the web interface shows each pipeline stage live. Evaluated honestly: the citation check has let **zero** invented sections through in every evaluation, but choosing the *right* sections is the weak part — on the published IL-TUR statute-identification benchmark it scores macro-F1 11.8 on the test split (9.8 on dev) against 28.1 for the best supervised model.

A2. The problem it solves

1. **The IPC → BNS transition.** From 1 July 2024 the BNS applies to offences committed on or after that date. Offences before it stay under the IPC, even if tried later. Section numbers were renumbered almost entirely. A lawyer in 2025–26 handles both codes at once. Some sections were merged (IPC 378 theft + 379 punishment → BNS 303), some split, some dropped (IPC 377 "against the order of nature", IPC 497 adultery), and the same number means different offences in the two codes (302 = murder in the IPC, a religious-feelings offence in the BNS).
2. **LLMs invent legal citations.** Studies of general chatbots and even commercial legal research tools in the US found frequent hallucinated or wrong citations (Dahl et al. 2024, "Large Legal Fictions"; Magesh et al. 2024 on legal research tools). This project saw it directly: asked what BNS 303 covers, the model confidently said a bomb-threat offence. BNS 303 is theft. (CLAUDE.md, 2026-09-25 entry.)
3. **Privacy.** A lawyer types a real client's facts. A free cloud model may train on prompts. So the system runs entirely on the user's machine.
4. **Cost.** A college project with zero recurring cost: no paid API, no paid hosting.

Who it is for: a lawyer or law student doing first-pass research on a criminal matter. **It is not** a judge, not legal advice, and not a replacement for reading the bare act.

A3. The worked example (learn this by heart)

Input: *"The complainant parked his motorcycle outside his house at night. In the morning it was gone. CCTV shows the accused pushing it away down the lane and riding off. The accused had no permission to take it and did not return it."* Offence date **15 August 2024**.

1. **Router** (no AI): 15 Aug 2024 ≥ 1 Jul 2024 → **BNS**.
2. **Intake**: the text has fewer than 3 court-order markers, so it is a fact statement and passes through untouched (no LLM call).
3. **VRAM handoff + guardrail**: Ollama is told to unload the LLM (6.5 s); BGE-M3 loads on the GPU; the best judgment similarity is above 0.576, so the case goes ahead (10.9 s, most of it loading the embedder).
4. **Retrieval** (0.5 s): statute search plus "precedent votes" rank BNS sections. Node 1 receives **14 candidates**: 303 Theft (**#1**), 317 Stolen property, 324 Mischief, 238, 331, 309 Robbery, 329, 316, 127, 103, 129, 330, 106, 345. The judgment pool Node 3 will need is fetched now too, while the embedder still holds the card.
5. **Node 1** (LLM) proposes **BNS 303**, with the reason "moved the complainant's motorcycle out of his possession without consent with the intent to take it dishonestly". Its fact profile says an offence is disclosed and there is one accused, so the joint-liability rule (which needs two or more acting together) adds nothing, and no rule adds a hurt section.
6. **Node 2** (graph lookup, no AI): `bns:303` exists, is classed as an offence (chargeable) → **verified**. Nothing rejected, so the retry edge does not fire.
7. **Node 2b** (LLM, two passes, order reversed): both passes quote the facts verbatim for the ingredients and name no missing ingredient → **supported** (green).
8. **Node 3** and **Node 5** run in parallel:
   - Node 3: BNS 303 has **0** direct precedents in the graph (no BNS theft case has reached an appeal court in the corpus), so it follows the verified mapping **BNS 303 ← IPC 378, 379** and returns three distinct theft judgments via IPC 379, from the Telangana, Himachal Pradesh and Delhi High Courts (similarity 0.686, 0.676, 0.658). Before the 2026-10-15 fix all three slots went to companion copies of one Telangana appeal; the near-duplicate check now keeps one per appeal.
   - Node 5 (pure SQL): BNS 303 direct, 0 judgments → **suppressed**. Via IPC 378: 11 judgments (8 allowed, 3 dismissed). Via IPC 379: 158 judgments (76 allowed, 1 partly allowed, 76 dismissed, 5 other). Labelled "not a conviction rate".
9. **Node 4** (LLM): one counter-argument against BNS 303 — the prosecution must prove *dishonest intention* (animus furandi); unauthorised movement alone does not establish it. Its fact quote ("The accused had no permission to take it and did not return it.") is found verbatim in the facts, it targets a charged section and cites no precedent label, so the final verification pass marks it **verified**.
10. The punishment card shows **up to 3 years, 1–5 years on a repeat conviction** (read from BNS 303(2)'s own words), and from the BNSS First Schedule: cognizable, non-bailable, triable by any Magistrate (the Schedule's second row for 303(2), property worth under ₹5,000 returned or restored, is community service, non-cognizable and bailable).

Time: **about 71 seconds** live on the laptop (unload 6.5 s, guardrail 10.9 s, retrieval 0.5 s, LangGraph 52.9 s). (`app/samples/motorcycle_theft.json`, field `timings`)

A4. The architecture in one picture

```
OFFLINE, built once
  statutes (IPC JSON, BNS gazette PDF) ─┐
  SC judgments (PDF, 2023-26)          ├─► clean ► chunk ► metadata ─► BGE-M3 ─► ChromaDB (4 collections)
  HC judgments (NyayaAnumana subset)   ┘                                  │
                                                                          └─► citation graph (NetworkX): sections, cases, edges
  IPC⇔BNS mapping (575 rows, reviewed)  ─► graph "maps_to" edges
  First Schedules (CrPC, BNSS)          ─► punishment / bail table
  outcomes (from graph edges)           ─► SQLite

RUNTIME, per case
  facts + offence date
   ► Stage 0  Offence-Date Router              (plain function)
   ► intake    court order? reduce it to facts (LLM, only if procedural)
   ► retrieval statute search + precedent votes, fused (RRF)   [BGE-M3 on GPU]
   ► guardrail nothing close enough? → "insufficient data" + review list, no LLM
   ► [hand the GPU from BGE-M3 to the LLM]
   ► LangGraph:
       Node 1  Statute Identifier (LLM, JSON)  → candidate_sections
       Node 2  Verifier (graph lookup)          → verified_sections / unconfirmed
          └─ retry edge: unconfirmed and retry_count < 2 → back to Node 1 with the rejection
       Node 2b Applicability judge (LLM ×2 + fixed rules) → supported / doubted
       Node 3  Precedent matcher  ┐ parallel
       Node 5  Outcome analytics  ┘ (SQL only)
       Node 4  Defence counter-arguments (LLM) + final verification pass
   ► FastAPI ► web page (or Streamlit), verified vs. doubted vs. rejected drawn differently
```

The single most important design idea: **the LLM only ever proposes; deterministic code decides what is real.** The router, the verifier, the rules, the analytics and every final check are ordinary code. The model is used where language understanding is unavoidable (reading facts, judging fit, writing arguments) and nowhere else.

Part B — How it works, stage by stage

B1. Stage 0 — the Offence-Date Router (`src/pipeline/router.py`)

- Plain Python, no AI. `route(date)` returns `"ipc"` if the date is before **2024-07-01**, else `"bns"`. A future date is rejected.
- **Boundary days** (both pinned by tests and by two cases in `broad_coverage.json`): an offence on **30 June 2024 → IPC**; on **1 July 2024 → BNS**. It is the date of the *offence* that decides, never the FIR, charge sheet or judgment date (BNS s358 saves proceedings for earlier offences).
- **Strict date parsing**: only ISO `YYYY-MM-DD` reaches the router. `08/09/2024` is 8 September in India and 9 August in the US, and a guessed date is a guessed code, so the router refuses anything ambiguous.
- The transition date lives in one place (`BNS_TRANSITION_DATE`, default `2024-07-01`); every later step takes the era the router returns, so no other code interprets the date.
- Why deterministic: every later step depends on it. A wrong code poisons everything downstream (wrong statutes, wrong numbers). A date comparison can be tested exhaustively; a model cannot.
- **Straddling periods**: if an offence ran across the transition (e.g. a fraud from June to August 2024), `route_period` runs the pipeline **once per code** and warns the user. (This came from the hardening pass, register row 6.)
- The web page reads the date out of the facts with regular expressions only (`src/api/dates.py`): day-first Indian format, never guessed. If there is no date, several dates, or only a month, the page **asks** instead of guessing.
- Test: Phase 4 gate — 10 cases × 3 repetitions = **30/30** routed correctly, and the runner also checked that retrieval touched only that code's collections.

B2. Intake — reading court orders (`src/pipeline/intake.py`)

- Problem found in Phase 6: real inputs are often court orders full of procedure ("learned counsel", "bail", "u/s 438 Cr.P.C."). That vocabulary dragged retrieval toward procedural sections and the model charged IPC 438 because the order mentioned *anticipatory bail under Section 438* — which is the CrPC, not the IPC.
- Fix: a deterministic test (≥ 3 of 22 procedural markers — "learned counsel", "petitioner", "bail", "u/s", "Cr.P.C", "charge-sheet", "Hon'ble", "heard", "public prosecutor"…) decides if the input is an order. Only then, one LLM call restates the alleged facts in 3–8 sentences (schema: `alleged_facts` ≤ 1,200 characters, then `facts_stated`), dropping procedure, section numbers and case numbers. The restatement must share ≥ 60% of its content words (4+ letters, stop-words removed) with the original or it is discarded and the original is used (protection against invented facts). Field order matters: asked for the flag first, the model said "no facts" for 8 of 10 orders.
- `nofacts`: if the order states **no allegation at all** (adjournment, bail conditions), the system declines with `no_facts_stated` instead of charging procedure. It declines only when the flag **and** an empty extraction (< 20 characters) agree — a single boolean never erases the rest of an answer. Unlike the guardrail path, no nearest sections are listed: for procedural text they would be summons and custody offences.
- Plain fact statements never trigger intake (0 of the project's fixtures do).

B3. Retrieval (`src/retrieval/`)

Embedder: BAAI/bge-m3, 1024-dimensional, fp16 on the GPU, cosine similarity. Chosen by a bake-off (see C6).

Four ChromaDB collections: `ipc_sections`, `bns_sections`, `judgments_pre2024` (read as "IPC-era"), `judgments_post2024` ("BNS-era"). Judgments are routed by the **code they apply**, not the decision date — because an IPC appeal decided in 2025 is still IPC law, and mixing it into the BNS collection would recreate the "same number, two codes" error.

Index-only views (they exist only in the index, never replace corpus text):

- *Sub-section views*: BNS 3 "General explanations" is 4,765 characters; common intention is a 194-character sub-section 3(5). Embedded whole, it is buried. Each sub-section gets its own vector.
- *Counterpart views*: civictech's IPC text has no Illustrations; the gazette BNS text does, and Illustrations read like case facts. So each IPC section also gets a vector of its verified BNS counterpart's text.

Two signals, fused:

1. *Statute search*: facts vs. section text. The top 60 sections are taken from a pool of 240 vectors (views collapse back to their section, so the pool is larger than the list).
2. *Precedent votes*: find the 60 most similar judgment chunks **per judgment collection**; keep each **judgment's** best-scoring chunk (one vote per judgment, so one long judgment whose paragraphs all resemble the facts cannot outvote five different courts); each judgment adds its similarity score to every section it cites. Reason: case facts resemble other case facts far more than they resemble abstract statute definitions.
    - Which judgments may vote, by era: an **IPC** query uses `judgments_pre2024`, and only judgments whose code is IPC. A **BNS** query uses both collections: BNS judgments vote directly; IPC judgments vote **only through the verified mapping** (their IPC section is translated to its BNS counterpart; an IPC section with no verified row casts no vote). A judgment naming both codes never votes, because its numbers cannot be attributed to one code.

- Fused with **Reciprocal Rank Fusion**, `score(s) = 1/(60 + statute_rank(s)) + 1/(60 + vote_rank(s))` (a term is dropped if the section is absent from that list) — the textbook k = 60, equal weights, not tuned on the test cases. Every alternative tried (k = 10, CombSUM, interleaving, IDF-weighted votes) was worse on the dev set.
- **Exclusion**: non-chargeable sections (definitions, general exceptions — read from `data/mappings/chargeability.json`) may neither rank nor receive votes. Reason: once the full mapping existed, IPC judgments citing definition sections voted BNS 2 "Definitions" into the top 3.
- **Worked numbers** (Phase 2 dev): precedent votes put the right section in the top 3 in 11/15 cases vs 9/15 by statute text alone; in the top 10, fused 13/15 vs statute text 14/15 — which is why Node 1 gets both lists (below).

Pair pooling (`ranker.pool_pairs`, switch `pair`): the IPC often splits an offence into a definition (503 criminal intimidation) and a punishment section (506). Statute search finds the definition (it has the words), votes find the punishment (courts cite "506 IPC"). Fused as unrelated sections, each held half the evidence and the offence vanished from the list. Pooling gives both their best rank. This fixed "threat to kill charged as attempt to murder" at the retrieval level.

What Node 1 reads: the *dual list* — the fused top 10 first, then any section of the statute-text-only top 10 not already listed: about 14–16 sections (14 in the worked example; 15.6 on average over 300 real orders), each with its title and a 300-character excerpt of its text. Non-chargeable sections are excluded. HyDE (restating the facts abstractly before the statute search) is in the code but **not used** at runtime — it measured no gain.

The guardrail (`src/pipeline/guardrail.py`): for each judgment collection of the era, take the single best chunk; if the best of those cosine similarities is below **0.576** (`MIN_JUDGMENT_SIMILARITY`), the facts are too far from anything in the corpus; the LLM is not called. Calibrated on 25 criminal fact patterns (lowest 0.595) vs. non-legal text (highest 0.557); 0.576 is the midpoint. Two lessons: (1) best *statute* similarity was useless (a rent dispute scored 0.537, above 10 of 25 real crimes); (2) **no threshold separates civil from criminal facts** (0.60–0.63, inside the criminal range), so civil cases are handled by the model's reading, not the guardrail. Since 2026-10-03 the guardrail never returns a bare refusal: it shows the nearest sections for review, because it once falsely refused a real mischief case (similarity 0.555).

B4. The VRAM handoff (an engineering detail examiners like)

The laptop GPU (RTX 3050 Ti) has **4 GB**. BGE-M3 at fp16 needs ~2 GB; qwen3.5:4b needs ~3.7 GB loaded. They cannot share the card. So, per case:

1. Ask Ollama to unload the LLM (`keep_alive: 0`) and **wait until** `/api/ps` **confirms** it is gone (Ollama otherwise keeps a model resident for minutes).
2. Load BGE-M3 on the GPU, encode the query, run statute search, precedent votes, **and prefetch the judgment pool Node 3 will need later** (because Node 3 runs after the LLM has taken the card back).
3. Free BGE-M3, then call the LLM. Measured: pre-generation time 36 s (query on CPU) → 18 s (GPU after eviction), identical output.

B5. Node 1 — Statute Identifier (`src/pipeline/nodes.py`)

- Input (two messages): a **system** prompt naming the code ("for this case it is the BNS: cite BNS section numbers ONLY"), and a **user** message with the offence date, the facts and the candidate list (`- BNS 303: Theft. <300-char excerpt>`).
- The system prompt's key instructions: read the facts as a court does when registering an FIR or framing a charge (allegations taken as true, prima facie); name only sections the accused could be **charged** under, never general exceptions, private defence or procedure; candidates are *suggestions* — most will be irrelevant, and a section not listed may be proposed (joint liability, abetment, attempt); "a case usually engages one to three sections".
- Output: **JSON constrained by a schema** (Ollama compiles the schema into the decoding grammar). Field order is deliberate — the fact profile comes **before** `sections`, so the grammar makes the model commit to "is this an offence, how many acted together, what injury" before it names sections:

```
{ "discloses_offence": bool, "why": "≤200 chars",
  "accused_count": 0-99, "acted_together": bool, "shared_plan_or_object": bool,
  "injury": "none|hurt|grievous hurt|death", "dangerous_weapon": bool, "harm_intended": bool,
  "sections": [ {"section": "≤40 chars", "reason": "≤200 chars"} ]   // at most 6 items
}
```

- Bounds are load-bearing: without `maxItems` / `maxLength` the model once listed sections until Ollama's 5-minute request limit (~4,000 tokens at ~16 tokens/s).
- Settings (`src/pipeline/llm.py`): `temperature 0`, `seed 0`, `think: false` (qwen3.5 is a "thinking" model and runs about half on CPU here, so hidden reasoning tokens would multiply latency), `num_predict` 1,024 as a backstop that **raises** an error rather than silently truncating, and `num_ctx` chosen as the smallest of 8K / 16K / 32K that holds the prompt (estimated pessimistically at 3 characters per token) — because Ollama silently **truncates** a prompt longer than `num_ctx`, and the model would then judge facts it never saw. Over 32K the call is refused (this is what the two IL-TUR test errors were).
- The injury definition in the prompt is the statute's own closed list of grievous hurt (with the code's own number of days: 20 in IPC 320, 15 in BNS 116(h)).
- **The model may not decline a case.** `discloses_offence: false` becomes a **warning** in the report; anything it proposed is still verified, and if nothing was, the nearest retrieved sections are shown for review. (Measured twice: a two-man phone snatching and a threat to burn a family's house were both called "civil disputes".)
- **Self-negation filter**: the model sometimes lists sections whose own reason begins "The facts do not disclose...". A regex anchored to the *start* of the reason (or "not applicable / not attracted / not made out") moves them to `withdrawn`, never verified; "the accused did not return it" is kept.
- **Rules that add proposals** (each then verified like any other): the joint-liability rule and the injury rule (B8), tagged `source: "rule"`.

B6. Node 2 — Verifier (`src/pipeline/nodes.py` + `src/graph/citation_graph.py`)

- **No AI.** `parse_citation` reads the model's text ("IPC 420", "Sec. 120-B", "120 B", "303(2)", "302/34", "379 read with 34" → two entries) and records which code, if any, the string *names*: `ipc`, `bns`, `other` (CrPC, BNSS, Evidence Act, BSA, or any "... Act") or none. A string naming both codes is `other` — it cannot be attributed. "120-B" is never cut to "120" (both are real IPC sections, so a wrong cut would verify).
- `verify_section(g, code, section)` asks: is `code:section` a node in the graph? **The code is required — there is no "either code" mode**, because the union of the two codes is exactly the trap (BNS 420 does not exist; IPC 420 does). A code the proposal names is honoured, never silently re-read as the era's code: "IPC 302" in a BNS case is rejected, not verified as BNS 302 (a different offence).
- The decision order for each parsed citation: (1) names another enactment → reject; (2) names the other penal code → reject with hint; (3) not a node of the era's code → reject ("BNS 420 is not a section of the ingested BNS (sections 1 to 358)", span read off the graph) with hint; (4) exists but **non-chargeable** → reject; (5) otherwise verified, with the graph's own title attached.
- Rejection reasons, as counted in the catch-rate table: does not exist in this code; belongs to the other code; real but not chargeable (a definition, general exception or punishment rule); another Act / malformed.
- **Chargeability layer** (`src/pipeline/chargeability.py`, data built by `scripts/build_chargeability.py` from the codes' own chapter structure): each section is `offence`, `liability` (34, 3(5), 149, 120B...), `non_chargeable` or `unknown`. For a section where only a sub-section creates liability (BNS 3 "General explanations": only 3(5) is common intention), a bare "3" is rejected with "cite that sub-section".
- Hints: if the model cites an IPC number in a BNS case, the rejection names the verified BNS counterpart ("its BNS counterpart is BNS 303"), read off the graph.
- Definition → punishment: a verified IPC definition (405) is charged under its punishing section (406).
- Sub-section: only the section is verified, so the model's sub-section choice is dropped unless a rule chose it (the model picked 109(2) for attempt to murder, 318(1) definitions, etc.).

B7. The retry loop (the LangGraph conditional edge)

- If Node 2 left anything unconfirmed **in the latest round** and `retry_count < 2`, the graph goes back to Node 1 with the rejections (`nodes.route_after_verify`). The round number *is* `retry_count`, so a retry that proposes nothing ends the loop instead of re-answering an older round forever.
- What made it work: the first version sent the feedback as a plain follow-up message, and the model re-proposed the same rejected `421` three times. Replaying its previous answer **as an assistant turn** and listing rejected numbers as **banned** fixed it: `421` ✗ → `378` ✗ (hint: BNS 303) → `303` ✓. (Phase 4 gate, 3/3.)
- The retry conversation, exactly as sent:

```
system    : <the Node 1 instructions for this code>
user      : Offence date … Facts … Retrieved candidate sections …
assistant : {"sections": [{"section": "421", "reason": "…"}]}        ← its own previous answer
user      : Your previous answer cited sections that could not be verified against the ingested BNS:
              - "421": BNS 421 is not a section of the ingested BNS (sections 1 to 358). …
            Never propose any of these again: 421. Already verified and kept: none; do not repeat
            them either. Propose correct BNS sections to replace the unverified ones, or an empty
            list if none applies.
```

- **Hints are graph lookups.** If a rejected number is a real section of the *other* code, the feedback names its counterpart under the verified mapping, with both titles — and adds "That is only correct if the original section was the right offence for these facts." Without that caveat a wrong guess (`421` for a theft) was faithfully translated into its real counterpart (BNS 320, concealing property from creditors), which then verified.
- Cap of 2 retries = at most 3 rounds, so a case can never loop forever.
- State keeps every round's proposals tagged with `round`, which is how the hallucination-catch rate is computed.

B8. Node 2b — the applicability judge (`src/pipeline/applicability.py`, `offence_rules.py`)

"Verified" only means *the section exists*. BNS 66/67/68 (sexual offences) verify for a fraud case. So a second check asks whether the facts support each section.

- One LLM call per pass covers **all** verified sections. Each section is shown with its ingested text (up to 1,200 characters; for an IPC punishing section such as 406, the definition it punishes — 405 — is prepended, because "shall be punished" names no ingredients; for a sub-section such as BNS 3(5), its own 194-character text).
- The model must, per section and in this order: copy up to **3** passages **word for word** from the facts (≤ 240 characters each), name the missing ingredient (or leave it empty), and say whether it applies. *Evidence before verdict* was chosen on the dev set from three measured variants: asking for the missing ingredient first made the model hyper-literal ("a fracture is not shown to be grievous hurt", which IPC 320 says it is).
- A quote counts only if, after normalising case, whitespace and curly quotes, it is an exact substring of the facts and at least 8 characters long; invented or paraphrased quotes are dropped and recorded as dropped. "None", "N/A", "all ingredients are present" are read as "nothing missing".
- **Supported** = the model says it applies **and** ≥ 1 grounded quote survives **and** no missing ingredient is named **and** no rule objects.
- **Two passes, section order reversed**, and "supported" requires **both** (self-consistency; the 4B model's verdict moved with the order of the list: one pass lost 0 relevant sections on one run and 4 on the next, with only the candidate list changed). On the saved dev runs, requiring both moved green-card precision from 17 right / 22 extra to 16 right / 8 extra.
- Every doubt shown carries a reason, never an empty one: the rule's objection, the missing ingredient the model named, "no passage of the facts could be quoted for it", or "the two readings of the facts disagreed".
- **Advisory, not a gate** (decided 2026-09-30): as a gate it removed the right charge in 4 of 15 dev cases for a precision gain of only 38% → 42%. So nothing is deleted; doubted sections are shown in amber with the reason.
- **Fixed rules**, each with an ablation switch:
  - *Headcount*: unlawful assembly/rioting need 5+ people (IPC 141–149, BNS 189–191); joint liability and conspiracy need 2+ (IPC 34, 120A, 120B; BNS 3, 61). Decided from `accused_count` (only when it is ≥ 1 — 0 means the facts do not say), never by the model.
  - *Liability rule*: 2+ accused acting together → propose IPC 34 / BNS 3(5); 5+ with a common object → IPC 149 / BNS 190. (Retrieval cannot find s34 — it ranked 146th — so the rule supplies it.)
  - *Injury rule*: only when the harm was intended, (injury grade, dangerous weapon) → the hurt section:

	no weapon	dangerous weapon or means
hurt	IPC 323 / BNS 115(2)	IPC 324 / BNS 118(1)
grievous hurt	IPC 325 / BNS 117(2)	IPC 326 / BNS 118(2)

  - *Death rule*: IPC 299, 300, 302, 304, 304A, 304B, 306 and BNS 80, 100, 101, 103, 105, 106, 108 need a death; if Node 1 reported the worst harm as none, hurt or grievous hurt, they are doubted with that reason.
  - *Attempt rule* (added 2026-10-15): for an attempt (IPC 307, 308, 511; BNS 62, 109, 110), a doubt whose **only** content is "death" ("death of the victim", "no death was caused"…) is overruled — an attempt is, by its own words, the case where death did not occur. Any other missing ingredient (intent, the act) still stands, and grounded quotes are still required.
  - *Grievous consistency*: if Node 1 reported grievous hurt and the judge supported the simple-hurt twin (IPC 324 / 323, BNS 115) but doubted the grievous form (326 / 325, 117) whose text is identical but for the

harm, the doubt is overruled.

- *Lesser-included*: a plain offence beside a **supported** aggravated form is shown as "included in" it, not double-counted. The pairs are those whose ingredients nest by the statute's own words — IPC: 418/419/420 ⊃ 417; 324/325 ⊃ 323; 326 ⊃ 323, 324, 325; 380/381/382 ⊃ 379; 392 ⊃ 379; 394 ⊃ 392, 379, 323; 407/408/409 ⊃ 406; 342 ⊃ 341; 448 ⊃ 447; 467/468 ⊃ 465. BNS: 117 ⊃ 115; 118(1) ⊃ 115; 118(2) ⊃ 115, 117; 304–307, 309 ⊃ 303; 127 ⊃ 126. A *doubted* greater offence absorbs nothing — if it falls, the lesser charge is what remains. Included sections get no precedents or analytics of their own.
- *Definition → punishment* and *chargeability* (B6) are rules too.

Every rule has a switch. `PRAMANA_ABLATE` (`src/pipeline/ablation.py`) takes a comma list of changes to switch **off**, so any change can be measured against the same code with only that change disabled. Never set in production.

switch	what turning it off does
`profile`	Node 1 is asked for sections only (no fact profile, so no decline warning and no liability rule)
`exclude`	non-chargeable sections are not removed from Node 1's candidates
`chargeability`	Node 2 does not reject non-chargeable sections
`judge`	Node 2b does not run; every verified section is a plain charge
`union`	(Node 1 two-ordering union — already off in code, `NODE1_UNION = False`)
`dual`	Node 1 gets only the fused list, not fused + statute-text
`punish`	Node 2 keeps an IPC definition (405) instead of its punishing section (406)
`intake`	a pasted order is not reduced to its alleged facts
`nofacts`	a fact-free order is analysed anyway instead of declined
`pair`	retrieval does not pool definition/punishment ranks
`lesser`	no "included in" marking
`injury`	no injury fields, no hurt-section rule, no death rule, no grievous consistency
`prima_facie`	the judge is not told to read facts as a court framing a charge
`subsection`	Node 2 keeps the model's own sub-section
`attempt`	an attempt doubted only for want of a death stays doubted

B9. Node 3 — Precedent Matcher (`src/pipeline/precedents.py`)

- A judgment is precedent for a charge **only if the citation graph has an edge** from it to that section. Similarity decides only the *order*. (So similarity alone can never make something a "precedent".)
- For a BNS charge it also searches the verified IPC predecessor's judgments — essential, because BNS precedent barely exists yet (BNS 303: 0 direct; IPC 378: 11, IPC 379: 158 in the current, audited graph). Each such precedent is labelled `via_ipc_predecessor` ("BNS 303 ← IPC 379").
- Ordering: judgments in the facts' neighbourhood first, by similarity of their best chunk; the rest by how many of their chunks cite the section ("most cited").
- Up to **3 per charge**, each with a 500-character excerpt; a judgment already used for another charge is not repeated; companion judgments (the same appeal filed under several numbers, differing only in a date or case number) are de-duplicated by the overlap coefficient of their 5-word shingles ≥ 0.6. Every precedent is also re-checked with `verify_case` (it cannot fail for a graph edge; checked anyway). Labels P1, P2… are assigned here, and Node 4 may cite only these labels.
- The judgment neighbourhood (the best chunk of each of up to 200 judgments per collection) is fetched before generation, while BGE-M3 still holds the GPU (the VRAM handoff).

B10. Node 5 — Outcome Analytics (`src/pipeline/analytics.py`)

- **Pure SQL, no LLM**, over `outcomes.sqlite` built offline from graph edges (2,699 cases, 16,493 case→section rows).
- Shows how appeals/petitions citing that section were **disposed of** (allowed / partly allowed / dismissed / other). The query is one `GROUP BY` over two tables (`cases`: 2,699 rows with their disposal; `cites`: 16,493 case→section rows): `SELECT c.disposal, COUNT(*) FROM cites s JOIN cases c ON c.doc_id = s.doc_id WHERE s.code = ? AND s.section = ? GROUP BY c.disposal`. The "allowed share" is (allowed + partly allowed) / (all decided, i.e. excluding "other").
- For a BNS charge the IPC predecessors' figures are shown too, labelled as such (worked example: BNS 303 has 0 judgments and is suppressed; IPC 379 has 158).
- **Not a conviction rate** — say this clearly if asked. An "allowed" appeal may be an accused winning or the State winning. NyayaAnumana's label is "petitioner succeeded", not "convicted".
- Suppressed below **5** judgments ("a percentage over three cases is noise presented as a statistic").
- Sentence ranges were **not** built: judgments mostly quote the *trial court's* sentence, which the appeal may have set aside.
- Runs in **parallel** with Node 3 in the LangGraph.

B11. Node 4 — Defence counter-arguments (`src/pipeline/defense.py`)

- One LLM call, up to 4 arguments, each targeting one charge, attacking an ingredient the facts do not clearly establish. Input: facts, each charge's statute text (up to 700 characters) with the judge's doubt beneath it ("Reviewer's doubt: ..."), labelled precedents `[P1] title (court, applies IPC 379): excerpt`, and (since 2026-10-15) the grievous-hurt definition (IPC 320 / BNS 116) when a charge turns on it (IPC 325, 326, 335, 338; BNS 117, 118, 122, 125).
- Output schema per argument: `against` (≤ 20 chars), `argument` (≤ 600), `fact_quote` (≤ 240, or ""), `precedents` (≤ 2 labels).
- **Final verification pass** (deterministic): the target must be a charged section; every precedent label must be one actually supplied; the fact quote must be verbatim; any other section number mentioned must verify. Failures are attached as `issues` and the argument is marked unverified (shown, flagged).
- Prompt rules added 2026-10-15 after reading demo output: an Explanation or Illustration widens an ingredient, never adds a requirement; one argument per ingredient.

B12. The web application (`src/api/`, `app/web/index.html`)

The interface (redesigned 2026-10-18). One principle governs it: **every graph and number on the page is data the system really produced or read from its own files; nothing is decorative or simulated.** The previous interface is kept at `/classic` as a fallback.

- **Three views**, in a dark side rail (keys 1 / 2 / 3; deep links `/#graph`, `/#system`, `/#sample=<id>`):
  1. **Workspace.** The welcome screen has a hero with the *live* citation graph around BNS 303 (theft): its IPC predecessors 378 and 379, the judgments that cite them, and the sections cited alongside. It is force-directed and draggable, with pulses running along real citation and mapping edges. Below it: corpus figures read from the build report (933 statute sections, 87,036 judgment passages indexed, 17,032 graph edges, and **0 unverified citations shown out of 889 proposed in 263 saved evaluation runs**), the pipeline diagram (deterministic code, local model and graph lookup drawn in different colours, with the retry loop and the insufficient-data exit), and the sample matters.
  2. **Citation graph.** An explorer for any section of either code. Selecting a section shows how many judgments cite it, its IPC↔BNS counterpart, the citing judgments and the co-cited sections. Selecting a judgment shows its court, year and disposal, with a summarise button.

3. **System & evaluation.** A dashboard computed when the page loads from `build_report.json`, the graph, and the saved evaluation runs:
    - corpus and graph sizes;
    - a table of every evaluation set (runs, proposals, rejected, leaked: always 0);
    - IL-TUR macro-F1, test 11.8 and dev 9.8, against the supervised 28.1 / 26.2 / 21.7, **labelled as below the state of the art**;
    - per-label F1 for the 14 most frequent labels;
    - Supreme Court judgments by year and High Court judgments by offence;
    - a "What it does not do yet" list.

- **During a live analysis:**
    - the pipeline track shows each stage's real duration;
    - the verification ledger (section × retrieved rank × proposed by × graph × facts) fills in from the progress events;
    - **"What retrieval found"** draws each candidate's fused RRF score as a bar, with its rank by statute text (T) and by precedent votes (V), and marks which ones the model proposed;
    - **"From candidates to charges"** is a funnel counted from the run itself.
- **In the report:**
    - **a citation graph of the matter**: the charges, their IPC↔BNS counterparts, the matter's precedents (ringed) and the other judgments citing those sections. Clicking a section opens its bare act; clicking a precedent scrolls to it.
    - **"How the search saw these facts"**: the facts, the candidate statute sections and the judgment passages, plotted as their real 1024-dimensional BGE-M3 vectors projected to 2-D with PCA (`src/pipeline/embedmap.py`). The page states how much variance the two axes keep (about 26% for the motorcycle case), so positions are approximate; the cosine shown on hover is exact. The map is computed while BGE-M3 still holds the GPU, before the VRAM handoff, and a failure there never fails the run.
- **Older samples:** the group-assault and job-scam samples were saved on 6 Oct, before candidate lists were stored, so their replays show no retrieval or embedding panels, and the page says so rather than inventing them.
- Everything is offline: no CDN, fonts and code vendored, and a test pins that the page makes no outside request. The four graph colours were checked with a palette validator for both themes.

API endpoints (`src/api/main.py`):

method + path	does
`GET /`	the web interface (`app/web/index.html`, served `Cache-Control: no-cache`)
`POST /analyze`	`{"facts", "offence_date": "YYYY-MM-DD", "offence_date_to"?}` → full report (used by Streamlit)
`POST /jobs` · `GET /jobs/{id}`	same body, run in the background; poll for `running / done / error`, the stage events so far, and the report
`POST /dates`	`{"text"}` → the offence dates written in it, each with the code it selects
`GET /samples` · `GET /samples/{id}`	the saved demo cases and their reports
`GET /section/{code}/{s}`	bare-act text, punishment, First-Schedule classification, counterpart (graph-checked; 404 if not a section of that code)
`GET /compare/{code}/{s}`	IPC ↔ BNS side by side with a word diff (verified mapping)
`POST /pdf`	raw PDF bytes (≤ 20 MB) → its text (digital PDFs; no OCR)

method + path	does
`POST /ask`	`{"report", "question", "history"?}` → an answer from the report only, every section mentioned checked in the graph
`POST /summarise`	`{"doc_id"}` → headnote of an ingested judgment, key quote kept only if verbatim
`GET /health`	Ollama reachable? graph and outcomes table present?
`GET /graph/{code}/{s}`	the citation-graph neighbourhood of one section (404 if it is not a section of that code)
`POST /graph`	`{"era", "sections", "doc_ids"}` → the neighbourhood of a report's charges and precedents
`GET /stats`	corpus, graph, IL-TUR and citation-check figures, read from the project's files
`GET /classic`	the previous interface, kept as a fallback

- Input limits: facts 20–12,000 characters; a question 3–600. A bad or future date → HTTP 422; Ollama down → 503 with a clear message.
- **One analysis at a time** (a lock): the 4 GB card holds either BGE-M3 or the LLM, so concurrent runs would fight over it; a second job reports "queued".
- **Report statuses**: `ok` (something verified); `no_verified_sections` (proposals, all rejected); `no_proposals`; `insufficient_data` (guardrail, or `no_facts_stated` for a fact-free order); `straddles_transition` (one run per code, with a warning). Every non-`ok` path except `no_facts_stated` still lists the nearest retrieved sections for review, labelled as unverified retrieval.
- **Live progress is real, not animated**: `src/pipeline/progress.py` emits start/done events with a summary of what each stage *actually produced*; the page draws the pipeline diagram and the "verification ledger" (section × retrieved rank × proposed by × graph × facts) from those events.
- Design: set like a law report (serif Spectral headings, IBM Plex Sans/Mono), one oxblood accent, light/dark themes; fonts are vendored so **the page makes no outside request** (a test pins this).
- Six saved samples replay instantly; any can be run live.
- **Lawyer tools** (each local, each graph-checked): bare-act drawer with punishment read from the section's own "shall be punished" clauses (and a separate repeat-conviction range); IPC↔BNS word diff; arrest/bail/court from the official First Schedules (BNSS: 465 rows; CrPC: 479 rows); PDF upload (PyMuPDF, no OCR); printable research memo; follow-up questions answered only from the report (every section mentioned is checked in the graph); precedent summaries (a key quote kept only if verbatim).
- A second frontend in **Streamlit** (`app/streamlit_app.py`, `run_demo.bat`) — the one CLAUDE.md originally specified — still works.

B13. Tech stack, with the reason for each choice

Layer	Tool (version)	Why this, and what was rejected
Language	Python 3.12.7	ecosystem; matched an existing CUDA torch install on the machine
Deep learning runtime	PyTorch 2.6.0 + CUDA 12.4	GPU embeddings; copied from an existing venv (same Python build) instead of a 2.6 GB download on a slow link
Embedder	**BAAI/bge-m3** via sentence-transformers 6.1	won the bake-off: recall@3 0.533 vs InLegal-Sbert 0.20; 8,192-token context; multilingual
Vector store	**ChromaDB 1.5.9** (embedded, file-based)	no server, free, supports list metadata + `$contains` filters; 4 collections
Citation graph	**NetworkX 3.3** (in memory)	no server; the graph is small (3,632 nodes, ~17K edges); pickled, rebuilt in seconds

Layer	Tool (version)	Why this, and what was rejected
Orchestration	**LangGraph 1.2**	explicit state machine: typed shared state, a conditional retry edge, parallel branches (Nodes 3 ‖ 5)
LLM	**qwen3.5:4b** (Q4_K_M, 3.4 GB on disk; Ollama reports 4.7B parameters, counting its vision encoder, which Pramana never uses) on **Ollama 0.40.2**	local, free, 256K context, stronger reasoning than gemma3:4b at the same size; gemma3:4b left no VRAM for the KV cache. Fallback: qwen3.5:2b
LLM client	plain **httpx** to Ollama's REST API	no SDK dependency; exact control over `format` (JSON schema), `think`, `keep_alive`
PDF extraction	**PyMuPDF**	measured on an 18-page judgment: 0.02 s vs pypdf 0.15 s vs pdfplumber 0.84 s, slightly more complete
Backend	**FastAPI 0.141** + uvicorn	async-friendly, job endpoints, static files
Frontends	plain HTML/JS (no framework, no CDN) + **Streamlit 1.64**	offline; the HTML page gives full design control
Analytics store	**SQLite** (stdlib)	Node 5 is SQL only
Tests	**pytest** — 413 tests	fake embedders/LLMs make most tests run in seconds without a GPU
Data handling	pandas (vanga parquet metadata)	
Hardware	Windows 11 laptop, RTX 3050 Ti 4 GB, 16 GB RAM	the constraint that shaped many decisions

What it does not use, on purpose: no cloud LLM (privacy), no fine-tuning (design rule), no paid API, no vector-DB server, no JS framework, no CDN.

B14. Every parameter in one table (for "what number did you use?")

stage	parameter	value	where	why this value
Router	transition date	2024-07-01 (offence date `<` → IPC)	`router.py`	the BNS commencement date
Intake	procedural markers	≥ 3 of 22	`intake.py`	0 of the project's fact statements reach 3
Intake	extraction overlap	≥ 60% content words	`intake.py`	rejects invented facts
Retrieval	embedder	BGE-M3, 1024-d, fp16, cosine	`embedder.py`	bake-off winner
Retrieval	statute pool / list	240 vectors → 60 sections	`ranker.py`	views collapse to sections
Retrieval	precedent pool	60 chunks per collection, 1 vote per judgment	`ranker.py`	one long judgment cannot outvote five courts
Retrieval	fusion	RRF, k = 60, equal weights	`ranker.py`	textbook default; alternatives measured worse
Retrieval	Node 1 list	fused top 10 + new statute-text top 10 (~14–16)	`graph.py`	votes crowd out textual matches
Retrieval	excerpt per candidate	300 characters	`graph.py`	
Guardrail	threshold	best judgment cosine 0.576	`guardrail.py`	midpoint of 0.557 / 0.595
LLM	model	qwen3.5:4b Q4_K_M via Ollama	`llm.py`	largest that runs usably on 4 GB
LLM	decoding	temperature 0, seed 0, `think: false`, JSON schema	`llm.py`	reproducibility, latency, parseable output

stage	parameter	value	where	why this value
LLM	context	smallest of 8K/16K/32K that fits; refuse > 32K	`llm.py`	Ollama silently truncates
LLM	reply cap	`num_predict` 1,024 (raises if hit)	`llm.py`	backstop only
Node 1	proposals	≤ 6 from the model (schema), plus at most 2 rule additions (liability, injury)	`nodes.py`	unbounded lists never ended
Node 1	reason / section strings	≤ 200 / ≤ 40 characters	`nodes.py`	
Retry	cap	`retry_count < 2` → at most 3 rounds	`nodes.py`	recovered 6/6; bounded latency
Node 2b	passes	2, order reversed, unanimous	`applicability.py`	order-sensitivity measured
Node 2b	quotes	≤ 3 per section, ≥ 8 chars, verbatim	`applicability.py`	
Node 2b	statute text shown	≤ 1,200 characters	`applicability.py`	
Node 2b	role	advisory (`JUDGE_GATES = False`)	`graph.py`	as a gate it lost right charges 4/15
Node 3	neighbourhood	200 judgments per collection, prefetched	`precedents.py`	VRAM handoff
Node 3	per charge	3, excerpt 500 chars	`precedents.py`	
Node 3	near-duplicate	5-word shingles, overlap coefficient ≥ 0.6	`precedents.py`	companion appeals
Node 5	suppression	< 5 judgments	`analytics.py`	"noise presented as a statistic"
Node 4	arguments	≤ 4, ≤ 600 chars, ≤ 2 precedent labels	`defense.py`	
Node 4	statute text shown	≤ 700 characters	`defense.py`	
API	facts length	20–12,000 characters	`main.py`	a pasted order fits
API	concurrency	one analysis at a time	`main.py`	one GPU

B15. The shared state (LangGraph `PipelineState`, `src/pipeline/state.py`)

The first eight fields are the project plan's schema with its exact names; the rest were added later, additively — none of the eight was renamed or repurposed.

field	type	holds	added
`case_facts`	str	the facts (after intake)	plan
`offence_date`	str	ISO date	plan
`candidate_sections`	list	**every round's** proposals, each tagged `round` and `source` (llm / rule) — the catch rate is computed from this	plan
`verified_sections`	list	passed Node 2; each with `code`, `section`, `subsection`, graph `title`, `role`, and `defined_in` when a definition was charged under its punishing section	plan
`precedents`	list	Node 3's judgments, labelled P1…	plan
`analytics`	dict	Node 5's disposal counts	plan
`defenses`	list	Node 4's arguments with `verified` and `issues`	plan

field	type	holds	added
`retry_count`	int	the current round	plan
`era`	str	"ipc" / "bns" — the only code Node 2 verifies against	Phase 4
`retrieved`	list	the ranked candidates Node 1 read	Phase 4
`unconfirmed`	list	rejected proposals with the reason (`why`) — the retry prompt quotes them, the report shows them struck through	Phase 4
`facts_profile`	dict	Node 1's answers about the facts	hardening
`applicability`	list	Node 2b's record per verified section (quotes kept/dropped, missing ingredient, passes, rule decisions, `included_in`)	hardening
`warnings`	list	e.g. straddling period, "the model considers this may not be an offence"	hardening
`precedent_pool`	dict	the prefetched judgment neighbourhood	Phase 5
`withdrawn`	list	sections Node 1 listed as *not* applying	2026-10-06

The LangGraph wiring (`build_graph(full=True)`):

```
START → statute_identifier (N1) → verifier (N2) ─┬─(open rejections & retry_count < 2)─→ statute_identifier
                                                 └─(otherwise)─→ applicability (N2b) → fan_out
fan_out ─┬─→ precedents (N3) → defense (N4 + final verification) → END
         └─→ analytics  (N5) ──────────────────────────────────────→ END
```

Router, intake, retrieval and the guardrail run **before** the graph as plain functions — they are deterministic (or, for intake, run before the GPU handoff) and need no state machine.

Part C — The data

C1. Every source, what it is, how it was obtained

Source	What	Size used	How obtained / trust
IPC	civictech-India `ipc.json` — 575 sections	~1 MB	GitHub, structured JSON. Only the operative paragraph of each section; **IPC 304B (dowry death) was empty upstream**
Official IPC consolidation (1997)	Legislative Department text (Goa Police copy), sha256-recorded	used for patches	filled 304B and **completed 80 truncated sections** (300's clauses and Exceptions, 506's threat-to-kill limb, 307's life-convict clause…) under strict rules (C4)
BNS	Ministry of Home Affairs gazette PDF, Act 45 of 2023, digitally signed	358 sections	**no trustworthy machine-readable BNS existed** — every GitHub copy was an unvetted hobby project; indiacode.nic.in timed out. So it was parsed from the gazette PDF
Supreme Court judgments	"vanga" open archive on AWS S3 (Mumbai), years 2023–2026, **English only**	828 MB of PDFs, 2,533 + 214 judgments	digital-born PDFs with a text layer (no OCR). Regional-language tars deliberately skipped
NyayaAnumana	Hugging Face, gated (accepted terms + token). The 2020–2024 single-judgment test file	1,781 judgments kept from 31,028 HC + SC rows	filtered to theft / hurt-assault / cheating / criminal breach of trust; read from the zip in place
IPC↔BNS mapping	built in-project: 575 IPC rows → BNS (574 reviewed in September; 304B → 80 added 2026-10-06 after 304B was patched into the corpus)	539 bridges, 36 "none"	proposed by a scorer, **every row reviewed** (by Claude, against gazette text — **not by a lawyer**)
First Schedules	BNSS (MHA gazette) and CrPC (CRPF copy, amended to 2013)	BNSS 465 rows / 288 sections; CrPC 479 rows / 395 sections	parsed by column position; cognizable / bailable / triable by
IL-TUR	ACL 2024 benchmark, gated; only LSI + PCR dev/test splits	153.6 MB	for evaluation only; the 1.2 GB train splits were never downloaded
Hanno-Labs indian-ipc-statute-identification	ungated CC-BY-4.0, 14,876 Patna HC orders with sections masked	sampled 30–60 at a time	the substitute external benchmark while IL-TUR access was pending

C2. Final corpus sizes (`data/processed/build_report.json`)

corpus	records
`statutes_ipc.jsonl`	575 sections (one section per chunk)
`statutes_bns.jsonl`	358 sections, every one, every marginal-note title
`judgments_pre2024.jsonl` (IPC-era)	83,918 chunks (SC + 37,313 NyayaAnumana HC chunks)
`judgments_post2024.jsonl` (BNS-era)	3,118 chunks
total	**87,969 chunks**

Index: `ipc_sections` 1,113 vectors (sections + counterpart views), `bns_sections` 667 (sections + sub-section views), judgments ~84K + ~3K. ChromaDB on disk: 2.6 GB.

Citation graph: **933 section nodes** (575 IPC + 358 BNS), **2,699 case nodes**, **16,493 case→section `cites` edges**, **539 `maps_to` edges** (IPC→BNS).

The striking fact about BNS data: only a handful of judgments apply the BNS (6 BNS-only and 22 naming both codes at the first build; still few by 2026). No BNS theft case had reached the Supreme Court. That is why Node 3's IPC-predecessor bridge is not optional — it is where nearly all post-2024 precedent comes from.

C3. Chunking and metadata

- **Statutes: one section per chunk** (rule from the project plan), plus index-only sub-section views.
- **SC judgments: by paragraph/holding, never fixed token counts.** Paragraphs are found by their numbers — with the rule that a paragraph number must be followed by whitespace (otherwise dates like `25.09.2015` at a line start opened 1,741 phantom paragraphs). Oversized "paragraphs" from failed splits (687 over 8,000 chars, largest **397,044**) are re-split at sentence boundaries above 4,000 chars.
- **NyayaAnumana: runs of whole sentences, ~1,200 chars**, because the source has no paragraphs at all (each judgment is one line, paragraph numbers stripped).
- **Every chunk has**: source, court, date (None for NyayaAnumana — its filename year is only a lower bound), `sections_involved`, doc id, collection.
- Validated against a fixed schema (`src/ingest/schema.py`); the writer re-reads its own line count from disk, and the gate cross-checks file counts against the build report (after a killed build once left a 0-line file that still "validated").

C4. Cleaning the text — the defects found and fixed

These are the problems an examiner may probe; each was found by **reading** data, not by automated checks.

1. **Ligature padding**: PDFs render `fi` as one glyph plus a space (`fi led` = "filed"). Hit **8.6% of judgment chunks** and exactly the important words (notification, definition, specified). Repair consumes exactly `len(expansion) - 1` spaces, and must run **before** whitespace collapsing.
2. **Running headers**: "SUPREME COURT REPORTS", party names, judge names inside **17%** of chunks. Stripped with a case-sensitive party-name rule so real prose naming the parties survives.
3. **Margin letters A–H** (SCR pinpoint letters) in 7.7% of chunks — stripped.
4. **BNS gazette glued words** ("ThisAct maybe called") — the PDF's own kerning, reproduced by every extractor; repaired by rule, never by guessing wording.
5. **BNS section heads without a space** (`104.Whoever`): a regex requiring `\.\s` silently lost 8 sections. Fix: each head must be exactly one more than the last.
6. **BNS titles are marginal notes** in a column that mirrors left/right by page. The first parser used the wrong coordinate (`x1` instead of `x0`), lost 11 titles **and glued them into other sections' text**, and a page-wrap rule gave 3 sections their neighbour's title (s97 "kidnapping" got "selling child for prostitution"). Rebuilt to assign notes by **document order**. All 358 titles now correct.
7. **IPC truncation**: civictech carries only the operative paragraph for ~80 sections. Completed from the official 1997 text **only where civictech's words are exactly the start of the official text (≥ 95%)**, because the 1997 text is *older* than civictech for anything amended since; the build fails if upstream changes under a patch.
8. **Bombay HC lost letters** after "f" ("frst", "ofence") — 140 repairs derived from the corpus's own vocabulary, each read.
9. **Devanagari mojibake** — 5–7 chunks, all inside Hindi quotations; a PDF font limitation, left as a warning.
10. **NyayaAnumana duplicates**: 1,863 exact duplicate rows; 49 SC rows that were copies of vanga judgments (detected by best-single-document containment ≥ 0.35).
11. **Per-court page furniture** in NyayaAnumana ("Downloaded on…", "Signature Not Verified Digitally Signed By…").

C5. Extracting section citations — the hardest data problem

The citation graph is only as good as the case→section edges, which come from regex extraction of "Section N of the IPC" from judgment text.

- **Membership is not provenance.** CrPC 378 (appeal against acquittal) and IPC 378 (theft) share a number; so do CrPC 406/407 vs. IPC 406/407, CrPC 482 vs. IPC 482. **7,035 CrPC citations** were sitting in the corpus as IPC sections. Fix: read the words after the citation; drop it if it names the CrPC, the Evidence Act, "the Code", or any capitalised "X Act".
- **"120-B" must not become "120"** (conspiracy vs. concealing a design — both real, so a wrong edge looks verified).
- **Sub-sections leaked as sections** (`303(2)` produced s2).
- **Civil judgments**: a judgment naming *no* penal code was validated against both codes, turning every civil "Section N" into a penal edge (1,214 civil SC judgments). Now: no code named → no penal sections; and judgments naming no penal code are dropped from the corpus (1,682).
- **The big audit (hardening pass)**: reading 100 random edges showed **37 were not penal-code sections (63% precision)** — headnotes name the Act *before* the sections ("Evidence Act, 1872 – ss.4, 112"), bare "Section 482" is the CrPC, `S.2587/2011` is a suit number. After the fix and a corpus-derived "bare-citation trust" table (482: 2 penal uses vs 3,044 other), a **fresh** 100-edge sample was **96% correct**. Edges went 24,475 → 16,514.
- **Judgments citing both codes**: edges are taken only from citations that name their code.

C6. The embedder bake-off (and why "domain model" lost)

- Plan said: use InLegal-Sbert (InLegalBERT, pre-trained on 5.4M Indian legal documents). A 2025 study (arXiv 2507.18455) had found general embedders match or beat it for Indian prior-case retrieval, so it was **measured, not assumed**.
- 15 hand-written fact patterns, each masked to its own code, recall@3 of the right section:

model	dim	R@1	R@3	R@5	MRR	peak VRAM
bhavyagiri/InLegal-Sbert	768	0.133	0.20	0.333	0.223	0.56 GB
BAAI/bge-m3	1024	0.067	**0.533**	**0.60**	**0.295**	3.26 GB (fp32)

- InLegal-Sbert showed **hubness**: the same generic sections (IPC 443, 278; BNS 18, 52) topped almost every query.
- Gotcha: `law-ai/InLegalBERT` loaded through sentence-transformers does **not** error — it silently gets an untrained pooling head. Only `bhavyagiri/InLegal-Sbert` is the trained sentence model.
- Locked before mass ingestion, because changing embedder means re-embedding everything.

C7. "Training" — there is none, and why

Nothing was trained or fine-tuned. This is the most likely trick question.

- The system is **retrieval-augmented generation**: a pre-trained embedder (BGE-M3) and a pre-trained LLM (Qwen 3.5 4B) are used as they are; knowledge comes from the retrieved statutes and judgments, and correctness comes from deterministic checks.
- Why not fine-tune: (1) the project rule "RAG, not fine-tuning" — the fix for a quality problem is better retrieval or prompting; (2) the 4 GB GPU cannot fine-tune a 4B model in any practical way; (3) laws change (they just did) — a fine-tuned model memorises old section numbers, whereas retrieval just needs the new statute ingested; (4) fine-tuning would not remove hallucination, while a graph check removes it structurally.
- **Be careful with one claim.** The project's founding notes say a fine-tuned Indian legal model (AALAP) scored worse than its base model on the AIBE bar exam. We could **not** confirm that number in the AALAP paper (arXiv

2402.01758 compares against gpt-3.5-turbo). Do not quote it as a fact; say "we chose RAG on design grounds and our measurements supported it".

- What *was* "learned" from data, without training: the guardrail threshold (calibrated), the embedder choice (bake-off), the bare-citation trust table (counted from the corpus), the chargeability table (from the codes' chapter structure), the First Schedules (parsed), the mapping (proposed by an overlap scorer, then reviewed).

C8. The IPC↔BNS mapping

- Scorer: **overlap coefficient** `|A∩B| / min(|A|,|B|)` over content words. Two earlier scorers failed: `SequenceMatcher` on sorted words matched theft to selling-a-child; Jaccard scored the *right* pair (IPC 378 → BNS 303) at 0.056 because BNS sections inline all Explanations and Illustrations (233 content words vs 13).
- Spot-checked against 21 known pairs: 19 right; both misses flagged `low_evidence`.
- Then **all 574 rows reviewed**: 97 in-scope first (89 confirmed, 8 corrected — e.g. 302 → 103, 379 → 303), then 477 more (408 confirmed, 33 corrected, 36 "none"). A "none" must be **proved by absence** (phrase search of the whole BNS: no "thug", no "order of nature", no adultery offence).
- Failure modes found: "Punishment for X" sections (mostly boilerplate) and two sections **with the same title** (the right answer was usually the near-tied runner-up). `low_evidence` missed some errors (308 → 110 and 316 → 92 were unflagged).
- Special cases: IPC 124A (sedition) → BNS 152 noted as a *redrafted successor*, so its precedent is persuasive only; IPC 420 is compared with BNS 318(4); 304B → BNS 80 was missing and added on 2026-10-06.

C9. Storage budget (the 20 GB discipline)

A self-imposed 20 GB ceiling (the disk actually had ~107 GB free), kept as discipline: smaller curated data embeds faster on 4 GB VRAM. Real numbers: venv 5.6 GB, ChromaDB 2.6 GB, raw data 1.2 GB (kept deliberately so extractor fixes could rebuild), processed 132 MB, Ollama model 3.4 GB, HF cache ~5 GB (half of BGE-M3's download is an unused ONNX folder). Regional-language judgment tars were never pulled; that alone avoided ~7× the planned judgment storage.

Part D — How it was built, and how well it works

D1. Timeline (from the git history: 100+ commits, 20 Sep – 18 Oct 2026)

Dates	Phase	What happened
20–21 Sep	**Phase 0, setup**	venv seeded by copying CUDA torch from an existing install (a 2.6 GB download on a ~30–700 KB/s link would take an hour); Hugging Face download failures diagnosed (Windows symlink privilege, stalled Xet transfer); embedder choice changed from "assume InLegal-Sbert" to "decide by bake-off"; Gemini was planned as primary LLM
22 Sep	**Phase 1, data**	statute + SC ingestion; ligature and header repairs; **cloud LLM dropped** — generation fully local (privacy of client facts)
24–26 Sep	Phase 1 cont.	15 test cases written; BNS title fix; truncated-file guard; mapping review (97 rows); NyayaAnumana ingested; **7,035 CrPC citations removed from IPC edges**; judgments routed by code applied
26 Sep	**Phase 2, retrieval**	BGE-M3 index; statute views; precedent votes + RRF; gate 11/15 dev (one short) + 10/10 held-out — accepted on 21/25 = 84%
26 Sep	**Phase 3, citation graph**	`verify_section` (code required), `verify_case`, precedents, mapping bridge; **adversarial gate 20/20**
27 Sep	**Phase 4, pipeline**	router, guardrail, Node 1, Node 2, retry loop; **gate PASS**: routing 30/30, adversarial retry 3/3. Data gaps filled: full 574-row mapping, 2026 judgments, broad-coverage test set
27 Sep – 4 Oct	**Hardening pass**	owner's rule: no Phase 5 until earlier issues closed. Found the **63%-precision graph edges** (fixed to 96%); chargeability; applicability judge (two passes); headcount/liability rules; ablations; held-out sets 2–5; research-driven methods tried and rejected (cross-encoder reranker, sparse hybrid, Node 1 union, HyDE); GPU query encoding
4 Oct	**Phases 5–7 MVP**	owner: "get the whole project done asap … a working mvp". Nodes 3, 4, 5, FastAPI, Streamlit, demo cases. **Phase 6** on an ungated external set (IL-TUR still blocked)
4–6 Oct	Intake; demo precision	court-order intake; lesser-included, injury, unanimous judge rules
6 Oct	**Web interface**	chat-style page, live pipeline view, evidence map, lawyer tools, law-report redesign
7–9 Oct	Accuracy pass	80 IPC sections completed from official text; fact-free orders declined; scored on fresh orders
9–14 Oct	**IL-TUR**	access granted; LSI dev 100 documents: macro-F1 9.8
14–15 Oct	Wrap-up and polish	launchers; precedent dedupe; repeat-conviction ranges; attempt rule; Node 4 grounding; samples refreshed
15–18 Oct	**IL-TUR test**	100 test documents, run once after all decisions: macro-F1 11.8
18 Oct	Final demo push	interface redesign, presentation, this guide revised, research paper completed

Working method (worth describing if asked "how did you manage the project"):

- one git branch per phase, merged only when that phase's written "Done when" gate was met;
- a single design record (`CLAUDE.md`) with non-negotiable rules and a "Known Gotchas" list — every corrected mistake was written down so it could not recur;
- **held-out test sets written before the change they test, and scored once** (the anti-overfitting protocol);
- every method change behind an **ablation switch** (`PRAMANA_ABLATE`) so it could be measured against the same code with only that change off;

- long evaluations made **resumable** (per-case checkpoint files), because the laptop's memory pressure killed them repeatedly.

AI assistance (be ready for this question). The code, data processing and documentation were developed with **Claude Code (an AI coding agent) as a pair-programmer**, directed by the project owner, who set the goals, constraints and decisions to delegate. Several "human" review steps (the mapping review, the gate reads) were explicitly delegated to the AI and are labelled as such in the data (`reviewer: "claude"`). Disclose this according to your institution's rules; the paper draft includes an acknowledgement.

D2. The gates of the seven phases

Phase	Gate (written before the work)	Result
1 Data	processed JSON validates; 20 random chunks read, none truncated/malformed	passed — and the read found 3 more defects that validation had missed
2 Retrieval	top-3 relevant in ≥ 12/15 test cases	11/15 (one short) + 10/10 held-out; accepted; later re-scored 12/15 judged
3 Graph	100% on 10 real + 10 fabricated section numbers	**20/20** (fabricated include BNS 420 and BNS 498A — real IPC numbers, the union trap)
4 Pipeline	10 cases route correctly every time; adversarial case triggers and survives retry	**30/30**, **3/3**
5 Full pipeline	motorcycle worked example end to end	done
6 Evaluation	real numbers in `stuff/docs/evaluation_results.md`, reproducible by one script	done (`scripts/eval_phase6.py all`, `scripts/eval_lsi.py`)
7 Frontend	motorcycle example works in the browser, no manual steps	done (`run_web.bat`)

D3. Metrics, explained simply

- **Recall@k (retrieval)**: fraction of test cases where a correct section is among the top *k* retrieved.
- **MRR**: mean of 1/rank of the first correct result.
- **"Right section charged"**: the case gets at least one correct section among its charges.
- **Precision**: of all sections charged, the fraction that are correct.
- **Micro-F1**: F1 over all (document, label) decisions pooled — dominated by frequent sections.
- **Macro-F1**: F1 computed per label, then averaged — every section counts equally, so rare sections matter as much as 302. IL-TUR LSI reports macro-F1.
- **Hallucination-catch rate**: of all citations Node 1 proposed, the share Node 2 rejected, and how many cases recovered within 2 retries; **leaked** = rejected citations that reached the charges (must be 0).
- **Definition = punishment** when scoring: IPC 378 (definition of theft) and 379 (punishment) count as the same offence on both sides.

D4. The results — every number you might be asked for

Citation verification (the core claim)

- Phase 3 adversarial set: **20/20**.
- Phase 6 hallucination-catch (`stuff/docs/evaluation_results.md` §3):
  - normal use (with retrieval), 30 real court orders, 94 proposals: **0 invented** — the model never proposed a non-existent number when shown candidates.

    - stress test (retrieval withheld), 15 dev cases, 68 proposals: **20 rejected (29%)** — mostly IPC numbers in BNS cases (420, 378–383, 465–468); **6/6 cases with a rejection recovered within 2 retries**; **0 leaked**.
- With retrieval context the model never took IPC-number bait ("Section 420" in a BNS-era complaint): 0/9.

Retrieval

- Bake-off: BGE-M3 recall@3 0.533 vs InLegal-Sbert 0.20.
- Phase 2: dev 11/15 strict → 12/15 judged; held-out 10/10, later 8–9/10 after data changes; the list Node 1 actually reads contains a relevant section in **26/27** cases (the miss is common intention, supplied by rule).
- External (Patna HC orders): a charged section in the list for **157/300 (52%)**.

End-to-end, project test sets (hand-written by Claude, not reviewed by a lawyer)

set	right section charged	precision
dev (15)	15/15	35%
held-out 2 (16)	14/16	58% (73% with doubted set aside)*
held-out 3 (10, criminal facts in civil-looking disputes)	6/10	31%
held-out 4 (12)	8/12	42% (59% with doubted set aside)
broad coverage (19)	16/19	68%

* Held-out 2 is shown as re-scored by `scripts/score_runs.py` (definition = punishment on both sides, `stuff/docs/evaluation_results.md` §5). The original scorer in `stuff/docs/hardening.md` gave 44% (57%) on the same run; quote 58% and know why the two differ.

- Across three held-out sets the judge doubted **0 of 31** correct charges.
- Out-of-domain probes: **10/10** declined correctly (civil, tax, labour, family → "no offence disclosed"; recipe, cricket, gibberish → "insufficient data" before any LLM call).

End-to-end, real court orders (Hanno-Labs, Patna HC)

sample	right section charged	precision	procedural sections charged
first 30	12/30 (40%)	14%	25 times
ext0040–59, intake on	13/20 (65%)	24%	3
ext0060–89, current code	13/30 (43%)	21%	1

IL-TUR Legal Statute Identification (the published benchmark)

	macro-F1 (all 100 labels)	micro P / R / F1
Pramana, dev split, 100 docs	**9.8** (11.6 over the 84 labels with support, 95% CI 8.6–14.3)	29 / 17 / 21
Pramana, test split, 100 docs (run once, 15-18 Oct)	**11.8** (13.9 over the 85 labels with support, 95% CI 10.9–17.2)	40 / 21 / 27
LeSICiN (supervised, published best)	28.1	
Hierarchical InLegalBERT (supervised)	26.2	
Hierarchical LegalBERT (supervised)	21.7	

- **Far below the state of the art.** The baselines are trained on IL-TUR's 42K-document train split; Pramana is zero-shot. For calibration only (a *different* Indian benchmark with different inputs — ILSIC, layperson queries, EACL Findings 2026, Table 3): zero-shot, **GPT-4.1 reaches 31.6 micro-F1 / 14.4 macro-F1**, Llama-3-8B 18.0

/ 7.0 and Gemma-3-12B 21.5 / 6.9. Pramana's 21 (dev) and 27 (test) micro-F1 with a 4B local model are in the same zero-shot range. The benchmarks differ, so this is context, not a ranking.

- **Dev and test agree.** Test is 2 points higher, but the confidence intervals overlap almost entirely: read it as the same performance on an unseen sample, not an improvement. The test split was run **once**, after every decision had been made on dev, with the same seed (20260927), code and settings.
- Where the score goes (dev → test): documents with no charge 30 → 33 (dev: most state no allegation, ~5 real facts wrongly declined; test: 24 declined by `nofacts`, 1 by the guardrail, 6 with no proposals, 2 errors); gold sections in the candidate list 34% (99/290) → 37% (95/258); minor companion offences rarely reach it (test: 504 0/5, 294 0/5, 324 0/4, 323 6/13); over-charging up the gravity scale (dev: 34, 307, 326; test: 34 ×15, 307 ×12, 149 ×8, 326 ×7); CrPC numbers among gold labels ~7% → ~5%.
- What it gets right on test: the grave, well-represented offences — IPC 302 F1 0.78 (18 correct), 279 1.0, 201 0.67, 304B 0.55, 306 0.50.
- **Errors**: dev 0 (3 VRAM failures re-run); test 2 (two documents over the 32K-token context this machine can hold; scored as empty). 16 test documents first failed on Ollama crashing (GPU out of memory) and were re-run with the same code, as on dev.

Speed

- ~1–3 minutes per live case (motorcycle ~71 s; knife ~145 s); IL-TUR documents 81 s on average on the test split (from a few seconds for a declined order to 538 s for the longest judgment).
- Pre-generation: 18 s with GPU query encoding (36 s on CPU).

D5. What was tried and REJECTED (shows scientific method — examiners like this)

idea	source of the idea	measured result	decision
InLegal-Sbert (domain embedder)	the original plan	R@3 0.20 vs 0.533; hubness	rejected
Hubness centring	literature	InLegal-Sbert 3→5, BGE-M3 8→6	rejected
IDF-weighted votes, RRF k=10, CombSUM, interleaving	IR literature	all ≤ RRF k=60	rejected
Cross-encoder reranker (bge-reranker-v2-m3)	COLIEE 2025 winners	dev recall@3 11 → 5 alone, 8 fused	rejected — rewards surface word overlap
BGE-M3 sparse / hybrid scoring	BGE-M3 paper (MIRACL +2.2)	top-3 11 → 7 fused	rejected — narrative facts vs abstract statutes share few words
Node 1 two-ordering union	COLIEE multi-LLM voting	relevant 20 → 22 of 31, precision 35% → 26%	switched off
HyDE (restate facts, then search)	literature	11/15 = no change; theft cases drifted	off
Judge as a gate (delete doubted)	—	precision 38→42% but right charge lost in 4/15	made advisory
Per-section judge calls	—	removed only 6/28 extras ("a small model says yes to anything in isolation")	kept batched
A companion-offence rule (504/323 questions)	IL-TUR dev analysis	macro-F1 on docs 36–100: 12.8 → 12.6	closed, not merged
Fine-tuning	—	—	out of scope by design

Part E — Problems, lessons, limits

E0. The challenges, grouped (with what each taught)

1. Hardware: a 4 GB GPU and 16 GB of RAM

- **Problem**: the embedder and the LLM cannot both fit in VRAM; the LLM itself does not fit fully (it runs ~52% CPU / 48% GPU), so it is slow.
- **Solution**: explicit VRAM handoff, verified eviction, prefetching Node 3's data while the embedder holds the card; fp16 embeddings (54–65 chunks/s vs ~10 at fp32; 84K chunks in ~21 min instead of ~2.5 h).
- **Problem**: long evaluations were **killed** repeatedly when free RAM fell under ~1 GB (Ollama ~4 GB + eval process ~4.7 GB + desktop apps).
- **Solution**: resumable runners with per-case checkpoints; `.bat` files to run outside the IDE; recording every interruption instead of hiding it.
- **Lesson**: constraints are design inputs. Several "clever" choices (a 4B model, prefetching, RRF instead of a reranker) exist because of the hardware.

2. A slow, unreliable network

- 18–700 KB/s, variable minute to minute; DNS failures for Ollama's blob host for ~2 hours; Hugging Face's Xet transfer stalled while plain HTTPS worked.
- **Lessons**: measure the host you are using, do not trust a quoted speed; Ollama pulls resume, Hugging Face downloads **never** resume (by design); disable symlinks on Windows (`HF_HUB_DISABLE_SYMLINKS`, not the `_WARNING` variant); set `HF_HUB_OFFLINE=1` or a model load can hang for 10 minutes on a metadata check.

3. Legal data is messy in ways automated checks do not see

- Ligatures, headers, glued words, wrong titles, truncated sections, CrPC numbers posing as IPC numbers, suit numbers posing as sections, duplicated rows, a 397,044-character "paragraph".
- **Lesson (the single most repeated one)**: *automated checks confirm the shape; only reading the content shows it is wrong.* Schema validation, zero duplicates and zero encoding errors all passed while 8.6% of chunks had broken words and 37% of graph edges were wrong.
- **Lesson**: audit precision with a **fresh** random sample — the sample that shaped a fix cannot also certify it (63% → 96% was measured on a new sample).

4. The two codes, renumbered

- Same number, different offence; merged and split sections; definitions vs punishment sections in the IPC but not the BNS.
- **Lessons**: never validate a citation against the union of both codes; never *score* retrieval against the union either; route judgments by the code they apply, not by date; treat a definition and its punishing section as one offence when scoring.

5. The small LLM's behaviour

- Unbounded JSON never stops → schema bounds. Retry feedback ignored → replay as an assistant turn + banned list. Lists sections it has itself ruled out → self-negation filter. Hyper-literal applicability judging ("a fracture is not shown to be grievous hurt") → prima facie framing, quotes-first prompt, deterministic rules. Verdicts change with the order of the list → two passes reversed. Picks wrong sub-sections → cite the verified section. Runs are not reproducible run to run even at temperature 0 → re-run demos twice, and score on enough cases to see noise.
- **Lesson**: the model is good at reading and writing, unreliable at bookkeeping. Move every decision that *can* be a rule into code; keep the model for language.

6. Verified ≠ applicable (the central quality problem)

- A graph check proves existence, never fit. The system over-charges: extra sections that exist but do not apply (precision 20–45% on most sets).
- Mitigations: applicability judge (advisory), headcount/death/attempt/injury/lesser-included rules, chargeability filter. Precision on dev rose modestly (33% → 39%); "doubted" is a useful signal (on IL-TUR dev, 135 of 220 wrong charges were doubted vs 29 of 71 right ones).
- **Lesson**: in legal AI, the asymmetric costs matter. Missing a real offence (or falsely declining a case) is worse than an extra flagged section, so the judge informs rather than deletes.

7. Evaluation honesty

- Hand-written fixtures flatter the system (60–100%) vs real orders (40–65%) vs the benchmark (macro-F1 11.8 test, 9.8 dev).
- The benchmark (IL-TUR) was gated and access took weeks; a substitute ungated set was used meanwhile and labelled as such.
- **Lessons**: write held-out sets before the change; score once; compare against the same code with only the switch flipped; report setbacks (the broad-coverage setback is in the record as "a SETBACK, recorded as is").

8. Safety: false declines

- Two real cases were wrongly declined (a threat to burn a family alive called "a civil dispute"; a mischief case refused by the guardrail).
- Fix: **no bare refusal on any path** — when the system thinks there is no offence or too little data, it still shows the nearest sections for review.

E1. The numbers card (memorise these)

what	number
Transition date	**1 July 2024** (offence date decides)
Sections	IPC **575**, BNS **358**
Judgments in corpus	2,699 (918 SC 2023–26 that apply a penal code + 1,781 HC from NyayaAnumana), every one a graph node
Chunks	**87,969**
Graph	933 section nodes, 2,699 case nodes, 16,493 cites edges, 539 maps_to edges
Mapping	575 IPC rows: 539 bridges, 36 none
Embedder	BGE-M3, 1024-d; bake-off R@3 **0.533 vs 0.20**
LLM	qwen3.5:4b, 3.4 GB, local Ollama
GPU	RTX 3050 Ti, **4 GB**
Guardrail	best judgment cosine **0.576**
Retry cap	**2** retries (3 rounds)
Judge	2 passes, unanimous, advisory
Graph gate	**20/20**
Routing gate	**30/30**
Edge precision	**63%** → **96%** after the audit fix
Hallucinations leaked	**0** in every evaluation
Stress test	20 of 68 proposals rejected, 6/6 recovered

what	number
IL-TUR LSI test (dev)	macro-F1 **11.8** (9.8) vs **28.1** supervised SOTA
Tests	**413** passing (as of 2026-10-15)
Time per case	~1–3 min
Cost	**₹0 recurring**

E2. Limitations (say these before you are asked)

1. **Precision**: it proposes too many sections; the judge flags many but not all.
2. **Not state of the art** on IL-TUR statute identification (zero-shot vs supervised).
3. **Narrow corpus**: Supreme Court 2023–26 plus a High Court subset filtered to theft/hurt/cheating/breach of trust. Offences outside those families have fewer close judgments.
4. **Very little BNS case law exists**, so post-2024 precedent comes almost entirely through IPC predecessors — persuasive, and not always identical law.
5. **Legal review**: the mapping, the fixtures' answer keys and the gate judgements were done by an AI, not a lawyer.
6. **Analytics are appeal disposals**, not conviction rates; no sentence ranges.
7. **Not deterministic** run to run despite temperature 0.
8. **Speed**: minutes per case on a laptop.
9. **English only**; PDFs without a text layer (scans) are not read (no OCR).
10. **Prior-case retrieval** (Node 3) is not evaluated on a benchmark (IL-TUR PCR was downloaded but not run).

E3. Future work (good answers to "what would you do next?")

- Lawyer review of the mapping and the test keys; then a lawyer-graded evaluation of Node 4's arguments.
- Run IL-TUR PCR for Node 3; extend the corpus to more High Courts and offence families.
- A precision layer trained on charge data — allowed only if the project's no-fine-tuning rule is revisited; a small classifier over (facts, section) pairs is the obvious candidate.
- Larger local model when hardware allows (7–8B fits a 8 GB GPU), or qwen3.5:2b for speed.
- OCR for scanned FIRs; Hindi input.
- Sentence extraction that ties each sentence to the court that imposed it and the final disposal.

E4. Glossary

Legal

- **IPC** — Indian Penal Code, 1860; the substantive criminal code until 30 June 2024.
- **BNS** — Bharatiya Nyaya Sanhita, 2023 (Act 45 of 2023); in force from 1 July 2024.
- **CrPC / BNSS** — Code of Criminal Procedure 1973 / Bharatiya Nagarik Suraksha Sanhita 2023 (procedure). **BSA** replaced the Evidence Act.
- **First Schedule** — the CrPC/BNSS table saying, for each offence, the punishment, whether it is cognizable (police may arrest without warrant), bailable, and which court tries it.
- **Section / sub-section / Explanation / Illustration** — Explanations and Illustrations clarify or widen a section; they are not extra requirements.
- **Definition vs punishment section** — IPC 378 defines theft, 379 punishes it; the BNS merged them (303).
- **Common intention** — IPC 34 / BNS 3(5): several people acting together in furtherance of a shared intention are each liable.
- **Unlawful assembly / rioting** — 5 or more persons (IPC 141–149; BNS 189–191).
- **Grievous hurt** — the closed list in IPC 320 / BNS 116 (fracture, endangering life, etc.).

- **Lesser-included offence** — a plain offence contained in an aggravated one (cheating 417 within 420).
- **Prima facie** — on the face of it; the standard at the charge-framing stage.
- **Precedent** — a decided judgment applying the section.
- **Disposal** — how an appeal/petition ended (allowed, dismissed…).
- **SCR** — Supreme Court Reports.

Machine learning / IR

- **RAG** — retrieval-augmented generation: retrieve relevant text, give it to the LLM.
- **Embedding** — a vector representing a text's meaning; similar texts have close vectors (cosine similarity).
- **Hubness** — in high-dimensional spaces some points are nearest neighbours of almost everything; here, the same generic sections topped every query.
- **RRF** — reciprocal rank fusion: combine rankings by summing 1/(k + rank).
- **Cross-encoder reranker** — a model that scores a (query, document) pair jointly; slower, often more accurate; here it was worse.
- **Self-consistency** — ask more than once and combine answers.
- **Zero-shot** — no task-specific training.
- **Ablation** — switch one component off to measure its contribution.
- **Held-out set** — test data not used while building; scored once.
- **Macro / micro F1** — see D3.
- **fp16** — 16-bit floating point; half the memory of fp32.
- **Quantisation (Q4_K_M)** — storing LLM weights in ~4 bits so a 4B model is 3.4 GB.
- **JSON-schema constrained decoding** — the runtime only allows tokens that keep the output valid against the schema.
- **LangGraph** — a library for building LLM workflows as state machines (nodes, edges, conditional edges).

E5. Running the project (and what to do if the demo breaks)

- Start: double-click `run_web.bat` → opens `http://127.0.0.1:8000/`. It starts Ollama if needed.
- Before a demo: close Chrome tabs and VS Code (memory); open each sample once; run your live case once.
- If a live run fails at Node 1: Ollama is not running or the model is missing → `ollama list` must show `qwen3.5:4b`. Saved samples still work without Ollama.
- If the page looks old after an update: hard refresh (Ctrl+F5).
- If anything in the new interface misbehaves on stage: open `http://127.0.0.1:8000/classic`, the previous interface, which runs the same pipeline.
- Rebuild from raw data: `build_corpus.py` → `build_index.py` (~25 min GPU) → `build_graph.py` → `build_outcomes.py`. Tests: `venv\Scripts\python.exe -m pytest tests -q`.
- Benchmark: `scripts\eval_lsi.py --split dev --n 100` (~2.5 h, resumable).

Part F — Question bank

Answer out loud, then check. **Bold** marks the phrase that should be in your answer.

F1. Basics

1. **What is Pramana?** A local research assistant for Indian criminal law: from facts and an offence date it picks the code, proposes sections, **verifies every section against a citation graph built from the real statutes**, checks the facts support them, finds precedents, drafts defence arguments and shows appeal outcomes.
2. **Why the name?** *Pramāṇa* is Sanskrit for "means of valid knowledge / proof" — the system's point is that every citation is proven against data.
3. **Who is the user?** A lawyer or law student doing first-pass research. Not a client, not a judge.
4. **What input does it need?** A description of the facts, and the offence date (typed, or read from the text; the page asks if unclear). It can also take a pasted court order or a PDF.
5. **What does it output?** Charges marked verified/supported, doubted (with reason), included-in, or rejected; precedents; counter-arguments; outcome counts; punishment and bail; a printable memo.
6. **Does it need the internet?** No. Everything runs on the laptop; no API key, no account.
7. **What does it cost?** Nothing recurring. All tools and models are free and local.
8. **What is the BNS and why does it matter here?** The Bharatiya Nyaya Sanhita, 2023, replaced the IPC for offences on or after **1 July 2024**, and renumbered nearly every section. Lawyers now work with both codes.
9. **Give an example of renumbering.** Theft: IPC 378/379 → BNS 303. Murder: IPC 302 → BNS 103; BNS 302 is a different offence (wounding religious feelings). Cheating: IPC 420 → BNS 318(4).
10. **Walk me through the worked example.** (A3) motorcycle, 15 Aug 2024 → BNS → 303 retrieved #1 → proposed → verified → supported → precedents via IPC 378/379 → argument on dishonest intention.

F2. Motivation and design choices

11. **Why not just use ChatGPT?** It invents or mixes up section numbers (our own model said BNS 303 was a bomb-threat offence), it does not know which code applies by date, and a lawyer's client facts would leave the machine.
12. **What is the single most important design idea? The LLM only proposes; deterministic code decides what is real.**
13. **Why RAG instead of fine-tuning?** Laws change (they just did) — retrieval needs the new text ingested, a fine-tuned model would need retraining; fine-tuning does not stop hallucination, a graph check does; the 4 GB GPU cannot fine-tune; and the project rule is that quality problems are fixed by better retrieval or prompting.
14. **Did you train any model? No.** Pre-trained BGE-M3 and Qwen 3.5 4B are used as they are. What was "fitted" to data: the guardrail threshold, the embedder choice (bake-off), the citation-trust table, the mapping (scored then reviewed).
15. **Why local generation only?** Privacy: a lawyer types a client's real facts, and free cloud tiers may train on prompts. Also no key, no cost, works offline.
16. **Why a 4B model and not something bigger?** It is the largest that runs at usable speed on a 4 GB GPU (and even it runs half on CPU). 7B+ models don't fit.
17. **Why Qwen and not Gemma or Llama?** gemma3:4b left no VRAM for the context cache on this card; qwen3.5:4b is similar size, stronger at reasoning, has a 256K context, and supports JSON-schema output well through Ollama.
18. **Why is the router not an AI?** It is a date comparison; making it a model would add risk with no benefit, and every downstream step depends on it. It is tested exhaustively.
19. **Why LangGraph?** The pipeline is a state machine with a **conditional retry edge** and **parallel branches** (Nodes 3 and 5); LangGraph gives typed shared state and those edges explicitly.

20. **Why NetworkX and not a graph database?** The graph is small (~3.6K nodes, ~17K edges), fits in memory, rebuilds in seconds; a database server would add setup with no gain.
21. **Why ChromaDB?** Embedded and file-based (no server), free, supports metadata filters on list fields.

F3. Data

22. **Where does the data come from?** IPC: civictech JSON (+ official 1997 text for completions); BNS: the MHA gazette PDF; Supreme Court judgments 2023–26 from the open "vanga" S3 archive; High Court judgments from NyayaAnumana (gated HF dataset), a filtered subset; mapping and schedules built in-project.
23. **How big is it?** 575 IPC + 358 BNS sections; 2,699 judgments (918 Supreme Court + 1,781 High Court); **87,969 chunks**.
24. **Why not all of NyayaAnumana (700K cases)?** Disk and embedding time on a 4 GB GPU, and relevance: a curated subset (theft, hurt, cheating, breach of trust) matched the test cases.
25. **Why only 2023–2026 SC judgments?** They straddle the transition, giving both IPC and BNS judgments; earlier years add size, not BNS coverage.
26. **How did you get the BNS text?** Parsed the official gazette PDF — no trustworthy machine-readable BNS existed. Titles are marginal notes, which needed geometry-based parsing.
27. **How were judgments chunked?** By paragraph/holding (paragraph numbers), never fixed token windows; oversize splits re-cut at sentences; NyayaAnumana (no paragraphs) in ~1,200-char runs of whole sentences.
28. **Why not fixed-size chunks?** A legal holding cut in half loses its meaning; paragraph boundaries are the natural unit of reasoning in a judgment.
29. **What metadata does each chunk carry?** Source, court, date, sections involved, document id.
30. **What was the hardest data problem?** Extracting *which law* a "Section N" belongs to. CrPC and IPC share numbers; headnotes put the Act before the sections; suit numbers look like sections. The graph's edges were only **63%** correct until an audit; after the fix a fresh sample was **96%**.
31. **How did you map IPC to BNS?** An overlap-coefficient scorer proposed a BNS section for each of 574 IPC sections; every row was then reviewed against the gazette text: 538 bridges, 36 "none" (proved by phrase search, e.g. adultery, s377). IPC 304B → BNS 80 was added later (575 rows, 539 bridges).
32. **Who reviewed the mapping?** The AI assistant (Claude), against the official text, row by row — **not a lawyer**. Each row records `reviewer: "claude"`. A lawyer review is listed as future work.
33. **What is NyayaAnumana's label and why can't you use it as conviction rate?** 1 = the appeal/petition was allowed, 0 = rejected. It is petitioner success — a 0 can be a refused bail or a failed State appeal.
34. **How did you clean the PDFs?** Ligature repair (8.6% of chunks), header/footer and margin-letter stripping, glued-word repair for the gazette, title reconstruction, sourced patches for truncated IPC sections. (C4)
35. **Did you delete raw data after processing as planned?** No — deliberately kept (~1 GB): extractor bugs kept being found, and every fix needed a full rebuild; NyayaAnumana is gated and the link is slow.

F4. Retrieval

36. **Which embedding model and why?** BGE-M3, by a measured bake-off: recall@3 **0.533 vs 0.20** for the legal-domain InLegal-Sbert on 15 test cases.
37. **Isn't a legal-domain model supposed to be better?** That was the assumption; a 2025 study found otherwise for Indian case retrieval, and our measurement agreed. InLegal-Sbert showed **hubness** — the same generic sections topped every query.
38. **What are "precedent votes"?** The most similar judgment chunks vote (weighted by similarity, one vote per judgment) for the sections those judgments cite. Facts resemble other facts more than abstract statute text.
39. **How are the two signals combined?** Reciprocal rank fusion, k = 60. Alternatives were measured and were worse.
40. **What does Node 1 actually see?** ~14–16 candidate sections: fused top 10 plus statute-text top 10, each with an excerpt; definitions and general exceptions excluded.

41. **Why did you add sub-section views?** Common intention is a 194-character sub-section inside BNS 3's 4,765 characters; embedded whole it ranks 155th.
42. **What is pair pooling?** The IPC splits definition (503) and punishment (506); text finds one, votes find the other; pooling gives both the better rank so the offence is not lost.
43. **Why didn't you use a reranker?** Tried the standard cross-encoder (bge-reranker-v2-m3): recall@3 fell from 11/15 to 5/15. It rewards surface word overlap (a theft at a house ranked house-breaking first).
44. **Why not hybrid (keyword + dense)?** Tried BGE-M3's sparse scores: worse (11 → 7). Narrative facts and statute text share few words.
45. **What is the guardrail?** If the best judgment similarity is below 0.576, the LLM is not called and the user gets "insufficient data" plus the nearest sections for review.
46. **How was 0.576 chosen?** Midpoint between the highest non-legal input (0.557) and the lowest real criminal fact pattern (0.595).
47. **Can the guardrail tell civil from criminal? No** — civil facts score 0.60–0.63, inside the criminal range. That is the model's job; the guardrail only stops non-legal text.
48. **Why does retrieval miss common intention (s34)?** It is an abstract liability rule; the facts' dominant meaning is the violence. s34 ranked 146th. A rule supplies it from the number of accused acting together.

F5. Verification and the citation graph

49. **What is in the graph?** Section nodes (IPC, BNS), case nodes (judgments), `cites` edges (case → section, from extraction), `maps_to` edges (IPC → BNS, verified mapping only).
50. **What does "verified" mean exactly?** The section exists as a node **in the code that applies** to this case. Nothing more.
51. **Why must the code be specified?** Because the union of both codes is the trap: "BNS 420" doesn't exist but "IPC 420" does; checking against both would pass it.
52. **Can the LLM bypass verification?** No. The report's charges are built only from `verified_sections`; rejected ones go to a separate `unconfirmed` field and are drawn struck-through. A test pins that rejected citations never reach charges ("leaked = 0").
53. **What happens when a citation is rejected?** The reason is recorded; if retries remain, Node 1 is asked again with its previous answer replayed and the rejected numbers banned, plus a hint (e.g. the BNS counterpart).
54. **Why only 2 retries?** Enough to recover in every measured case (6/6), and it bounds latency; the cap guarantees termination.
55. **What did the Phase 3 gate test?** 10 real and 10 fabricated section numbers, including BNS 420 and BNS 498A (real IPC numbers), repealed IPC 13, and patched 304B — **20/20**.
56. **What is the hallucination-catch rate?** In normal use the model proposed 0 non-existent sections in 94 proposals. With retrieval withheld (stress), Node 2 rejected **20 of 68 (29%)**, all cases recovered within 2 retries, **0 leaked**.
57. **So the hallucination problem is solved?** The **citation-existence** problem is solved structurally. The **applicability** problem — real sections that don't fit the facts — is not; that is the main weakness.

F6. The LLM nodes, the judge and the rules

58. **How do you get reliable JSON from a 4B model?** Ollama's JSON-schema constrained decoding, with `maxItems` / `maxLength` bounds; temperature 0, a seed, thinking disabled, a token cap that raises an error instead of truncating.
59. **What is Node 2b?** The applicability judge: for each verified section, the model must quote the facts for its ingredients; a quote counts only if it appears verbatim. Two passes with the order reversed; supported only if both agree.
60. **Why two passes?** The 4B model's verdict changed with the order of the list; requiring agreement of both orders makes "supported" more trustworthy.

61. **Why is the judge advisory and not a filter?** As a filter it deleted the right charge in 4 of 15 dev cases for a small precision gain. Missing a real offence is the worse error, so doubts are shown, not enforced.
62. **Name the deterministic rules.** Headcount (5+ for rioting, 2+ for joint liability), liability (propose 34 / 3(5)), injury grade (→ hurt section), death (murder needs a death), attempt (attempt to murder needs no death), grievous consistency, lesser-included, chargeability (definitions are never charges), definition→punishment.
63. **Why rules instead of better prompts?** Counting people or checking whether someone died is bookkeeping, which a small model does unreliably and code does perfectly. Each rule has an ablation switch and was measured.
64. **Give an example of the model being wrong and a rule fixing it.** The judge doubted attempt to murder because "death" was missing — but an attempt is exactly the case where death did not occur. The attempt rule overrules only that objection.
65. **How does Node 4 avoid inventing case law?** It may only cite labels [P1]... of precedents actually supplied; a final deterministic pass checks each label, each quote (must be verbatim from the facts) and each section mentioned (must verify); failures are flagged on the argument.
66. **How are precedents chosen?** Only judgments with a graph edge to the charge; ranked by similarity to the facts; for BNS charges also via the IPC predecessor; near-duplicate companion judgments merged.
67. **What does Node 5 compute?** SQL counts of how appeals/petitions citing the section were disposed of, suppressed below 5 judgments. No LLM.
68. **Why are Nodes 3 and 5 parallel?** Both depend only on the verified sections and not on each other; LangGraph runs them as parallel branches.
69. **What is the "intake" step?** For pasted court orders: a deterministic marker test, then one LLM call to extract only the alleged facts, checked for word overlap with the original.

F7. Evaluation

70. **How did you evaluate?** Five layers: per-phase gates; hand-written dev and held-out sets (scored once each); out-of-domain probes; an external set of real Patna HC orders; and the published IL-TUR LSI benchmark.
71. **What is IL-TUR?** The Indian Legal Text Understanding and Reasoning benchmark (ACL 2024). LSI = legal statute identification: given facts, predict the IPC sections (100 labels).
72. **What did you score?** Macro-F1 **11.8** on 100 test documents (micro-F1 27) and **9.8** on 100 dev documents (micro-F1 21), against **28.1** for the best supervised model (LeSICiN).
73. **Why so far below?** The baselines are **trained on IL-TUR's 42K training documents** and its label conventions; Pramana is zero-shot with a 4B local model. Many documents state no allegation; minor companion offences (504, 323) rarely reach the candidate list; it over-charges grave sections (307, 326).
74. **Is that a fair comparison?** It is the standard metric on the standard split, so it is reported; but it compares a supervised classifier to a zero-shot system. For context only (a different benchmark): on ILSIC, zero-shot GPT-4.1 gets 31.6 micro-F1 and 8–12B open models 18.0–21.5; Pramana's 21 (dev) / 27 (test) is in that zero-shot range.
75. **Did you tune on the test set?** No. Decisions were made on dev; the test split was run once at the end (started 2026-10-15, finished 2026-10-18).
76. **What is your precision and why is it low?** About 20–45% depending on the set. The model over-proposes real-but-inapplicable sections, and the IL-TUR/charge-sheet labels themselves often list several companion sections the facts don't spell out.
77. **Your hand-written test sets were written by an AI — isn't that circular?** They flatter the system (60–100% vs 40–65% on real orders), which is exactly why the external set and the benchmark were added and are the numbers we lead with.
78. **How do you know results aren't noise?** Small sets are noisy — we say so: on 20 cases, one or two flips are within run-to-run variation (we saw a case flip with identical input). We report confidence intervals on IL-TUR (8.6–14.3 for the supported-label macro-F1) and avoid claims from one-case differences.

79. **What is an ablation and did you do one?** Turning one component off with everything else the same. Yes: every hardening change has a `PRAMANA_ABLATE` switch; e.g. the fact profile off lost common intention (recall 14 → 12) and raised extras (30 → 37).
80. **What did you NOT evaluate?** Prior-case retrieval on a benchmark (IL-TUR PCR); the legal quality of counter-arguments (no metric; read by hand); speed under load.

F8. Hostile and twisted questions

81. **"So it's just ChatGPT with extra steps?"** No — the model never decides what is real. A deterministic router picks the code, a graph lookup verifies every number, rules handle counting and logic, analytics are SQL, and a final pass checks the generated arguments. Zero invented sections have reached a report in any evaluation.
82. **"Your score is a third of the state of the art. Why should anyone use this?"** It is not competing as a classifier. It is a research aid whose citations are guaranteed to exist in the right code, which runs privately on a laptop, and which explains each section with the statute text, punishment, bail and precedent. On accuracy of section *choice* it is weak, and we say so.
83. **"If the model can't choose sections well, what's the point of verification?"** Verification removes the most dangerous failure — a confident, non-existent or wrong-code citation — so what remains is a reviewable list rather than a fabricated one.
84. **"Why should I trust an AI-reviewed mapping?"** You should treat it as unreviewed by a lawyer. Its method is documented: every change located by an exact phrase match in the gazette text, every "none" proved by absence. It is on the future-work list for lawyer review.
85. **"Isn't the guardrail threshold overfitted to 35 examples?"** It is a midpoint in a measured gap on 35 inputs, and it is only used to stop *non-legal* text; it never refuses a case outright (it always shows the nearest sections). We documented one false refusal and changed the design because of it.
86. **"Temperature 0 means deterministic, so why do your demos vary?"** We measured that it does vary (the same group-assault facts gave different extra sections on three runs), even with temperature 0 and a fixed seed. The likely cause is numerical: the model runs split across GPU and CPU, and floating-point results can differ slightly between runs, which can flip a near-tied token. We did not prove the cause; we handle the effect by re-running demo cases and by not trusting one-case differences in evaluations.
87. **"Your 'precedents' for BNS theft are IPC judgments — that's wrong law."** For offences re-enacted in identical terms (IPC 378 → BNS 303) the IPC precedent remains good guidance; the report labels it "through the IPC predecessor". For redrafted sections (124A → 152) the mapping notes that precedent is persuasive only.
88. **"Can a lawyer rely on this in court?"** No. It is a research aid; the page says so. Every section links to its bare text for the lawyer to read.
89. **"What if the offence date is wrong?"** Then the code is wrong; the router can only be as right as the date. That is why the page asks when it cannot read a single clear date, and handles periods straddling 1 July 2024 by running both codes with a warning.
90. **"Why not fine-tune — wouldn't 28 F1 be easy then?"** Possibly better on IL-TUR, but it would memorise one benchmark's label conventions and today's section numbers, cannot run on this hardware, and would not remove hallucination. It was excluded by design; we report the cost of that choice.
91. **"The LLM is 4B — isn't it too small for law?"** It is small, which is why the system leans on retrieval and rules; where it matters most (citations), its output is checked. The measured weakness (applicability, precision) is exactly where size shows.
92. **"How do you know the extraction of sections from judgments is right?"** We audited it: 100 random edges read by hand, 63% correct; fixed; a *new* random 100 read: 96%. Each audited string is now a test.
93. **"Didn't you just tune until the numbers looked good?"** The protocol prevents that: held-out sets were committed before the changes they test and scored once; setbacks are recorded (the broad-coverage set got worse after hardening, and that is in the report).
94. **"Your analytics show 70% of appeals allowed — so 70% acquittal?"** No. It is disposal of appeals/petitions — an allowed appeal can be a State appeal, a bail plea or a quashing. The page labels it "not a conviction rate".

95. **"What happens to my client's data?"** It never leaves the machine. No network calls at runtime except to the local Ollama server; the web page loads no external resources (a test pins this).
96. **"Who wrote the code?"** Built with Claude Code as an AI pair-programmer under the owner's direction. (Answer honestly and per your institution's policy; know the design reasons in this guide, because that is what an examiner will probe.)
97. **"What's the most embarrassing bug you found?"** The graph's edges were 37% wrong for weeks — CrPC procedure numbers and other Acts counted as IPC offences. It was found only by reading 100 edges by hand.
98. **"What would break if someone typed a Hindi FIR?"** The embedder is multilingual, but the corpus, prompts and checks are English; results would be poor. English only is a stated limitation.
99. **"Isn't 'verified' misleading to a lay user?"** That is why the page separates three levels: verified (exists), supported (the facts were quoted for it on both readings), doubted (exists but the check found a gap, with the reason) — and draws them differently.
100. **"Why not use Indian Kanoon's API?"** Paid/limited and an outside network call with case facts; the project's rules are zero cost and fully local.

F9. Engineering

101. **How do the embedder and the LLM share a 4 GB GPU?** They don't — they take turns: evict Ollama and confirm via `/api/ps`, load BGE-M3, encode and prefetch everything later stages need, free it, then generate.
102. **What did fp16 buy you?** ~5–6× faster indexing (84K chunks in ~21 min) and it fits the card.
103. **How is the live progress view done?** The pipeline emits start/done events with a summary of what each stage produced; the browser polls a job endpoint and draws them. Nothing is simulated.
104. **How many tests and what do they cover?** 413 pytest tests (counting parametrised cases) across ingestion, the pipeline nodes and rules, the API, retrieval, the graph and the evaluation code; fake embedders and scripted LLMs make the suite run in about 10 seconds without a GPU.
105. **How do you prevent a corrupted corpus?** The writer fsyncs and re-reads its own line count; the gate cross-checks every file against the build report; the build fails on mismatch.
106. **What's an example of a Windows-specific problem?** Hugging Face downloads failing with WinError 1314 (symlinks need admin) — fixed with `HF_HUB_DISABLE_SYMLINKS`; and Ollama models landing on C: because the service never saw the user-level variable.
107. **How long does indexing take?** ~25 minutes on the GPU at fp16.
108. **How do you handle a court order that is mostly procedure?** The intake step extracts alleged facts; if none are stated, it declines with `no_facts_stated` rather than charging procedural sections.
109. **What happens if Ollama is down?** The launchers start it; saved samples still work; a live run returns a clear 503.
110. **Where is the design record?** `CLAUDE.md` (rules, status, every gotcha) and `stuff/docs/` (one file per evaluation).

F10. Process and reflection

111. **What did you learn most?** Read the data — automated checks confirm shape, not content. And move every decision that can be a rule out of the model.
112. **What went wrong in the project?** The hardening pass improved the dev set but a held-out set got worse (19 → 17 relevant verified); the graph-edge bug; repeated eval runs killed by memory; three weeks of gated-dataset waiting.
113. **What would you do differently?** Audit extracted edges on day one; write the external evaluation earlier; secure benchmark access first; pick a GPU with 8 GB if possible.
114. **How did you avoid overfitting?** Held-out sets written before changes, scored once and then "spent"; same-code ablations; the test split run once at the end.

115. **Why are there two frontends?** Streamlit was the planned one; the HTML page was added for a professional, offline, fully controlled design with a live pipeline view. Both call the same API.
116. **What's novel here?** Not the components — the combination: date-routed dual-code retrieval for the IPC→BNS transition, code-scoped graph verification with a retry loop, precedent bridging through a reviewed mapping, and an honest zero-shot evaluation on a 4 GB laptop.
117. **What's the biggest risk if deployed?** Over-trust: a user reading "verified" as "applies". Mitigated by the three-level display and the bare-act links; a real deployment would need lawyer review and user testing.
118. **Ethical concerns?** Privacy (solved by local-only); automation bias (labelled research aid); data licences (public judgments; NyayaAnumana and IL-TUR accessed under their terms; Hanno-Labs CC-BY-4.0).
119. **If you had one more month?** Lawyer review of the mapping and keys; a precision model over (facts, section) pairs; IL-TUR PCR for Node 3; more High Court data.
120. **Summarise the project in one line. A zero-cost, fully local legal research assistant that can't cite a section that doesn't exist — and is honest about how often it cites one that doesn't apply.**

F11. New since 2026-10-15 (the test split and the fine detail)

121. **Test was 11.8 and dev 9.8 — so it improved?** No. The 95% intervals (10.9–17.2 and 8.6–14.3 over labels with support) overlap almost entirely; it is **the same performance on a different 100-document sample**. Nothing was changed between the two runs.
122. **How do you know you did not tune on the test split?** It was run once, after all decisions, with the same seed, code and settings, from a committed batch file (`scripts/run_iltur_test.bat`). The only re-runs were documents where Ollama itself crashed (GPU out of memory) — the same procedure as on dev — and the two documents too long for the machine stay as errors.
123. **Why did two test documents fail?** They needed ~53K and ~33K tokens; the context is capped at 32K because Ollama silently truncates anything longer, and judging a truncated judgment would be worse than an honest error.
124. **Why does the context size change per call?** The smallest of 8K/16K/32K that fits is used, because a larger context costs memory and time on a 4 GB card, and a smaller one would silently cut the facts.
125. **What exactly is in the retry message?** The model's own previous answer replayed as an assistant turn, the rejected strings with their reasons, a "never propose these again" list, the sections already kept, and a request for replacements or an empty list (B7).
126. **What does "one vote per judgment" protect against?** A single long judgment whose many paragraphs all resemble the facts would otherwise outvote five different courts.
127. **What does the judge see for IPC 406?** The text of 405 (the definition of criminal breach of trust) prepended to 406, because a punishment section names no ingredients.
128. **Name three things the model is never allowed to decide.** The code (date router), whether a section exists (graph), and whether five or more people were involved for rioting (headcount rule). Also: precedent membership (graph edges) and the final check on arguments.
129. **What is in the shared state besides the plan's eight fields?** era, retrieved, unconfirmed, facts_profile, applicability, warnings, precedent_pool, withdrawn — all additive (B15).
130. **Why would the analytics show IPC figures for a BNS charge?** BNS case law is almost non-existent (BNS 303: 0 judgments), so the verified IPC predecessors' figures are shown beside it, labelled as such — never merged into the BNS count.